

A FOUR-DAY BUILD, DOCUMENTED HONESTLY

NotebookLM, in *four days*.

A complete teardown of how one Next.js engineer rebuilt the NotebookLM experience end to end. Multi-provider AI, hybrid retrieval, self-critiquing deep research, two-voice audio, mind maps, flashcards, quizzes, encrypted credentials, streaming UI – without a separate Python backend, without a vector database, without a marketing budget.

DAYS TO BUILD	REPOS	PROVIDERS	STUDIO KINDS
4	1	12	8

WHAT'S INSIDE

Table of *contents.*



Sixteen chapters across roughly forty-five pages. Each chapter takes the room it needs — from the Monday-night architectural restraint that bought the week, through the encrypted-credential vault that lets a single user run twelve different AI providers from one settings page.

01	The thing I did not build	<i>The architectural restraint</i>	03–04
02	The stack in one breath	<i>Twelve decisions, one repo, one deploy</i>	05–07
03	Comparison	<i>FastAPI · LangChain · this stack</i>	08–10
04	UI architecture	<i>Three-pane shell, panels, modals, state</i>	11–13
05	Backend split	<i>Route Handlers vs tRPC, when to use which</i>	14–15
06	Authentication	<i>Better Auth, ownership, onboarding gate</i>	16–17
07	Ingest pipeline	<i>Parse · chunk · embed · store</i>	18–20
08	Multi-dimensional embeddings	<i>Four sibling tables, non-destructive</i>	21–22
09	Source selection	<i>Status state machine, readiness contract</i>	23–24
10	Three-stage retrieval	<i>Expand · hybrid · rerank</i>	25–27
11	Deep research	<i>Plan · search · critique · gap-fill · verify</i>	28–31
12	Streaming + AG-UI	<i>NDJSON, events, observable progress</i>	32–34
13	The studio pane	<i>Audio · mind map · flashcards · quiz · notes</i>	35–39
14	Multi-provider AI	<i>Twelve providers, encrypted at rest, one factory</i>	40–42
15	Citations	<i>How the AI references sources, tags them</i>	43–44
16	Thesis	<i>What I would build the same way every time</i>	45

PREMISE • THE DECISION THAT MADE THE WEEK POSSIBLE

01

The thing I *did not* build.

Before the stack, the non-decision. The most important call of the week was not which framework to choose. It was which one to refuse, before writing a single line of code.

My first instinct, because I have written a lot of Python, was to split this into two apps. A FastAPI service for the AI work — embeddings, retrieval, deep research, TTS orchestration. A Next.js front-end that talked to it. One week, two runtimes, two deploys, two sets of secrets, two CORS configurations, two test suites.

I threw that plan out on Monday night. The honest reasons:

THE AI SDK IS THE REASON

Vercel's **ai** package, version 6, makes every LLM call a one-liner. **generateObject**, **generateText**, **streamText** — all with streaming, tool calling, and structured output baked in, exposing a single uniform **LanguageModel** interface across every provider's SDK. Whatever I was going to gain from Python's ML ecosystem, I would lose three times over in glue code between two services.

Specifically, the AI SDK gives me four things I would otherwise have to hand-roll: token streaming with backpressure, structured-output parsing with Zod schemas, tool-call lifecycles, and provider portability that does not require rewriting the whole call site. None of those are hard to build, but each one is a day. Four days would have been the entire build budget.

PGVECTOR LIVES ON THE SAME POSTGRES

If I am already running Drizzle against Neon for users, notebooks, and messages — and I am — then adding a **vector(768)** column and doing distance queries is free. There is no separate vector database, no separate deploy, no separate billing line, no separate dashboard to learn, no separate backup story, no separate tail-latency profile to debug.

The right time to add a dedicated vector database is when pgvector stops scaling for the workload you actually have, not when a tutorial says you should. For tens of millions of vectors per node, with HNSW indexes, that day will not arrive in your first year. Probably not in your first three. By then your product has either succeeded enough to justify the operational cost of a second database or failed for reasons unrelated to retrieval performance.

THE EMBEDDING API IS HTTP

I was going to call Google's embedding endpoint either way. It does not care whether the caller is Node or Python. The supposed "ML ecosystem advantage" of a Python service evaporated the moment I asked what specifically Python would do for me that Node could not. The honest answer was nothing. Numpy is fast for arrays in memory; here I am sending text over HTTPS to a remote model and storing the returned floats in a database. The local computation is a rounding error.

The rule I would give anyone building a RAG app in 2026: do not split the backend unless you have already hit a real constraint you can name. "I know Python better" is not a real constraint. It is inertia.

What the monolith bought me

The cost of architectural restraint was zero hours. The benefit was structural and showed up everywhere across the build.

One mental model held in working memory. The whole app fits in your head when there is no service boundary. When I changed the schema, every query that touched it failed to typecheck, and the failures showed up *at compile time*, in the same file the change was made. There was no "did I update the FastAPI Pydantic model" question to ask. Drizzle is the contract.

One deploy. Push to GitHub, Vercel deploys. There is no orchestration to write, no Docker file to maintain, no Kubernetes manifest to tweak, no inter-service version skew to hunt down at 2am. The thing that goes to production is exactly the thing that ran on my laptop. There are deployment models with more flexibility than this. None of them are faster.

One auth surface. Better Auth holds the session. The session cookie travels with every request, including to the heavy AI route handlers. The user identity is available in the procedure context for free. There is no service-to-service auth handshake, no JWT introspection latency, no shared-secret rotation between services to remember.

One observability story. Logs flow to one place. Traces — when I add them — span one process. There is no distributed tracing context to propagate across an HTTP boundary. Errors appear once, with one stack trace, originating in the file that caused them.

What I gave up — and why it does not matter

The honest accounting must include the costs of the monolith, because there are some.

I cannot scale AI workloads independently of web traffic. If a user kicks off a deep research run that takes three minutes, that is three minutes of one Node lambda being held open. Vercel handles this fine up to **maxDuration: 300** on the route, and beyond that I would move to background jobs. But I cannot, today, scale "AI runtime" separately from "API runtime." For my user volume, this is not a constraint that has bitten.

I cannot put the AI service behind a different network policy. In some enterprise contexts, the AI work needs to live in a different VPC, or behind a different egress gateway, for compliance reasons. If that becomes the constraint, the right move is to extract the AI calls into a separate service. But that decision becomes load-bearing only when the constraint is named, not before.

The Python ML ecosystem is genuinely larger. If I needed to fine-tune a model, run inference locally on heavy frameworks, or use a Python-only library for something esoteric — I would lose. None of those are required for a product that orchestrates remote LLM APIs and stores embeddings in pgvector. The decision was load-bearing only against the alternative I was actually choosing between, not against every conceivable architecture.

The takeaway

Four months later, the same monolith now supports twelve AI providers, multi-dimensional embeddings, encrypted credentials with versioned keys, eight studio output kinds, a self-critiquing deep-research agent, and a streaming-UI protocol that paints every step of every long-running operation. Not one of those features required a second deploy target. Not one required leaving the typed-end-to-end Drizzle world. The architectural restraint that bought the week kept buying weeks every month after.

The cheapest decision in software is the one that lets you say no to a service boundary before it exists. The expensive decision is the one that creates a boundary you have to maintain forever in exchange for an option you may never exercise.

The stack

in *one breath*.

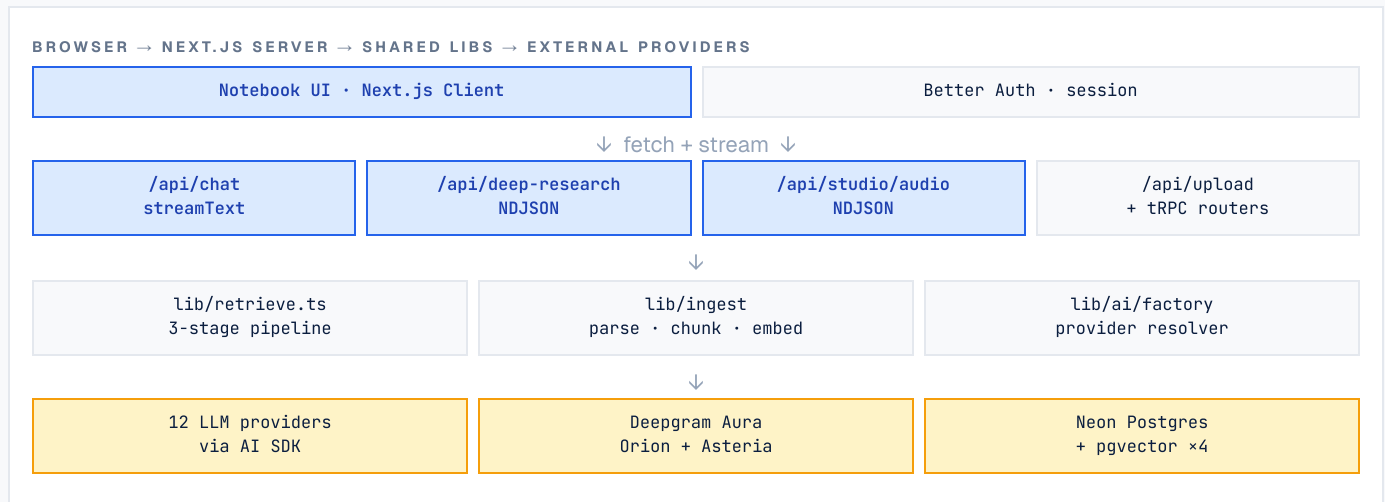
02

Every line below is a decision that earned its place. Nothing is here because it was trendy. Everything is here because removing it would make the build measurably worse. This chapter walks each layer top to bottom, names the alternative I considered, and explains the choice.

FRAMEWORK Next.js 15 App Router · Route Handlers · RSC · maxDuration 300 · Server Actions where useful	AUTH Better Auth Sessions · email/password · OAuth providers · server hooks for onboarding
DATABASE Drizzle + Neon Postgres Serverless HTTP driver · branching · pgvector extension · timestamps everywhere	VECTORS pgvector · 4 sibling tables 768 / 1024 / 1536 / 3072 dims · HNSW cosine indexes · provider-portable
AI LAYER AI SDK v6 ai · all 9 SDK adapters · streamText · generateObject · generateText · structured output	RPC tRPC v11 notebook · source · studio · provider · aiConfig · message routers
CHAT MODELS User-selectable, 12 providers Default Gemini Flash · Claude · GPT-4o · o1 · Grok · Llama · Mistral · Mixtral	EMBEDDINGS User-selectable, 6 providers Google · OpenAI · Mistral · Cohere · Voyage · Ollama · Together
TTS Deepgram Aura aura-orion-en + aura-asteria-en — one host each, MP3 over HTTPS	STORAGE S3/R2 + Supabase StorageService abstraction · uploads + audio output · presigned URLs
WEB EXTRACT @mozilla/readability Article extraction from arbitrary URLs · DOM-shape-aware	WEB SEARCH Exa + Tavily Pluggable provider order · fallback via SEARCH_PROVIDER_ORDER env

A simple way to read this list: each row names a *responsibility*, the *tool* that fulfils it, and a one-line *note* about how it was used. There are no overlapping responsibilities. Every concern lives in exactly one layer.

The architecture, as one picture



How the layers actually compose

The browser renders the three-pane workbench (sources, chat, studio). It speaks to the server in three ways: tRPC for non-streaming RPC, Route Handlers for streaming endpoints, and an upload endpoint for multipart bodies. Better Auth's session cookie travels with every request automatically.

The Next.js server is divided cleanly into Route Handlers and tRPC procedures. Route Handlers are streaming surfaces — chat, deep research, audio overview generation. They *are* the AI work, executed inline. tRPC procedures are everything else: notebook CRUD, source ingestion triggers, studio output listing, provider credential management. Both surfaces share the same database and the same auth.

The shared libraries are the moat. Three of them carry most of the weight. **lib/retrieve.ts** is the three-stage retrieval pipeline (229 lines). **lib/ingest** is the chunk/embed/store flow (split into chunk.ts, embed.ts, parse.ts). **lib/ai/factory.ts** is the provider-resolver that loads encrypted credentials, decrypts them, and returns the right AI SDK **LanguageModel** for the user's currently configured provider. Routes call libraries; libraries call providers. The shape is unidirectional and obvious.

The external providers are the LLM endpoints (12 of them, swappable per user), the TTS service (Deepgram, two voices), and the database (Neon Postgres with the pgvector extension and four sibling embedding tables for the supported dimensions). None of these own state in the application sense. State lives in Postgres.

Why this shape

Notice what is *not* in the diagram. There is no separate vector database. There is no separate Python service. There is no message queue. There is no Redis. There is no third-party orchestration layer. There is no LangChain. There is no separate auth service. There is no API gateway between client and server. There is no service mesh. The diagram is small because the moving parts are few.

Each absence is a deliberate non-decision. Each one was on the table at the start. Each one would have added a deploy, a config, a failure mode, and a reasoning surface. Each one was excluded because nothing in the requirements demanded it. When the requirements *do* demand it — when the load gets large enough, when the team gets multiple hands, when the operational story justifies it — adding any one of them is a single contained change. Until then, their absence is the load-bearing decision.

The dataflow for one chat message

To make the architecture concrete, here is what happens when a user types a question and presses send. Every layer above gets exercised in this one path.

1

The client posts to `/api/chat`.

The browser calls `fetch("/api/chat", { body: JSON.stringify({ notebookId, sourceIds, messages }) })`. The session cookie travels with the request. The body includes the user's selection of source ids — the chat is scoped to whatever sources are checked in the left pane.

2

The route handler authenticates and authorises.

Better Auth resolves the session from the cookie and yields a `userId`. The handler verifies that the user owns the notebook by joining notebooks on `userId`. If not, 404. If yes, the next step proceeds with confidence.

3

`retrieveForQuery` runs the three-stage pipeline.

Stage 1 expands the query into 2-4 phrasings via Gemini Flash. Stage 2 runs hybrid retrieval — pgvector cosine on the active embedding-dim sibling table, plus a regex pass on the chunk content table. Stage 3 reranks the top ~24 candidates with the chat model. Output: top-12 chunks with source title and content.

4

`streamText` produces tokens.

The chunks become a numbered list in the system prompt with explicit "cite every claim with `[^n]`" instructions. `streamText` opens an HTTP connection to whichever LLM provider the user configured. Tokens flow back and are streamed out via `result.toUIMessageStreamResponse()`.

5

`onFinish` persists the citations.

When the stream closes, the `onFinish` callback inserts a row into the messages table with the assistant text plus a JSONB `citations` column — an array of `{ n, chunkId, sourceId, title, snippet }`. The citations are frozen with the answer.

6

The client renders citations as live pills.

A regex on the streamed text matches `[^n]` and replaces each occurrence with a clickable superscript. Hovering the pill shows a tooltip with the source title and snippet. Clicking scrolls the left pane to the source or opens its modal.

What is special about this dataflow

Six steps. No service hops. No queue. No race condition between "save the message" and "stream the message" — the streaming returns through the same route that does the persisting. The same handler that sees the request sees the request close. The same database connection that loaded the chunks writes the message. There is no eventual consistency anywhere in the path.

For most "AI chat with citations" products, this single path involves 3-5 services, 2-3 queues, and a handful of callback contracts. Here it is one function. The function is **123 lines**. That number is not bragging — it is the receipt for the architectural decision two chapters ago.

The shape of this picture is the entire thesis: one server calls shared libraries calling external providers. The shared libraries are the moat — three-stage retrieval, the multi-dim embed pipeline, the provider factory. The route handlers are thin wrappers that stream. The UI subscribes.

COMPARISON • THE STACKS I DID NOT PICK

03

Why *this* stack, not the others.

Three other stacks were on the table on Monday. Each one has merits. Each one would have cost the week. This chapter walks each alternative honestly, then lays out the trade-offs side by side. The point is not to dunk on any of them; the point is to make the trade visible so you can run the same comparison for your own build.

Alternative one: FastAPI + Next.js

The classic split. Python on the back for AI work, Next.js on the front for UI. This is the architecture I would have built two years ago without thinking. It has real strengths: Python has the larger ML ecosystem, FastAPI ships with Pydantic-based request validation, the AI work runs in a runtime where async is well-understood and the libraries (LangChain, LlamaIndex, Chroma, sentence-transformers) are abundant.

The cost is real too. Two deploys means two CI pipelines, two secrets stores, two log streams, two version-skew problems. The CORS configuration sits between them. The auth context has to travel — usually as a JWT or as a session-introspection round-trip. The data model has to be defined twice — once as a Drizzle schema for the front, once as a Pydantic model for the back — and kept in sync by hand or by a code generator that introduces its own surface area. Streaming from Python to TypeScript involves SSE or WebSockets with reconnection logic on both sides.

For a product where the AI work is genuinely heavy — thousands of concurrent inference calls, custom model serving, GPU scheduling — the costs of the split are paid for by gains the monolith cannot match. For a product that orchestrates remote LLM APIs and stores embeddings in Postgres, the costs are paid for by nothing.

Alternative two: LangChain + Pinecone

The framework-first approach. LangChain provides retrievers, chains, and agent abstractions. Pinecone is a hosted vector database with strong tooling. Together they give you the "scaffold a RAG app in 200 lines" tutorial flow that everyone has seen.

The cost of LangChain's abstraction is that you lose visibility into what is happening. When the three-stage retrieval is opaque inside a `create_retrieval_chain`, you cannot tune it. When the deep-research loop is hidden behind `create_react_agent`, you cannot make the model self-critique with a quality threshold. The cleanest version of every concept in this build is shorter and more legible without LangChain than with it. The retrieval pipeline is 229 lines of plain TypeScript. The deep-research route is 713 lines that you can read top to bottom and understand. Inside LangChain, those same flows live behind 3-4 layers of abstraction that you have to dig through every time the behaviour does not match your expectation.

The cost of Pinecone is the operational footprint of a second database. You pay separately, you authenticate separately, you back up separately, you reason about its consistency model separately. None of that is hard, but all of it is cost. For a product whose users will not exceed 10M chunks in their first year, the cost is unjustified.

Frameworks are an answer to the question "I don't know how to do this and I want a path." For a builder who knows how to do it, a framework is mostly subtraction — fewer lines you write but more lines you have to read to understand what the framework is doing on your behalf.

Alternative three: serverless functions + managed everything

A third alternative was on the table: a Vercel + Supabase + Pinecone + Langfuse + LangSmith stack. Each component is best-in-class. Together they feel like the modern answer for a builder who wants to ship fast without owning infrastructure.

The cost shows up in two places. First, in the number of dashboards. Five managed services means five places to log in, five sets of API keys, five billing relationships, five sets of incident pages to monitor. Second, in the cost of *integration*. Each managed service has its own SDK, its own opinions about how data flows, its own auth model. You spend more time gluing them together than you would building the equivalent functionality yourself against a single Postgres.

There is a real point at which managed-everything becomes the right answer — when the product has scaled to multiple engineers and the cost of operating self-hosted services exceeds the cost of paying for managed ones. For a one-person build in week one, the math is exactly inverted.

Side by side

CONCERN	FASTAPI + NEXT.JS	LANGCHAIN + PINECONE	THIS STACK
Deploys	2 (back + front)	2-3 (app + Pinecone + worker)	1 (Vercel)
Vector store	FAISS / Chroma / Pinecone	Pinecone, separate billing	pgvector on the same Postgres
RAG framework	Custom or LlamaIndex	LangChain abstractions, opaque chains	Plain TS · 229 lines
Streaming	SSE or WebSocket bridge	Chain.stream() callbacks	NDJSON · streamText · ReadableStream
Provider portability	Each provider's SDK directly	LangChain wrappers, schema drift	AI SDK · uniform interface · 12 providers
Type safety	Pydantic + TS, two type systems	Loose, mostly dict-shaped	Drizzle + Zod end-to-end TS
Auth context	JWT or service-to-service	Whatever LangServe ships with	Better Auth cookie · same process
Build cost (1-person, week 1)	~10 days	~7 days, brittle	4 days
Ongoing cost	2 deploys, 2 secrets stores	3-5 services, multiple bills	1 Vercel + 1 Neon

The table is not a fair fight. Each alternative has scenarios where its trade-offs land correctly. But for a one-person, four-day build of a NotebookLM-shaped product, the rightmost column wins on every row that matters.

When each alternative is right

I want to be honest about the conditions under which I would have chosen differently, because builders read tables like the one on the previous page and conclude "the right answer is always the rightmost column." That is not the lesson.

FastAPI + Next.js becomes the right answer when the AI work needs to run in a runtime where Python's ML ecosystem is load-bearing. Local model inference. Custom embedding models trained on your domain. Heavy NumPy/PyTorch integration. GPU scheduling. None of those are required for orchestrating remote LLM APIs and storing their outputs.

LangChain becomes the right answer when the team building the product is large enough that *standardisation across builders* matters more than per-flow legibility. When you have 8 engineers each writing their own RAG pipeline, the framework's opinions are a coordination mechanism, even if each individual pipeline is harder to read than a plain-TypeScript version. For a one-person build, that benefit is zero.

Pinecone becomes the right answer when you cross the threshold of pgvector's headroom. That threshold depends on the workload — a rough rule is that pgvector with HNSW handles tens of millions of vectors per node with single-digit-millisecond p50 search latency. When you exceed that, the engineering of a separate vector database is worth it. Until then, the operational cost of a second database is wasted.

Managed-everything becomes the right answer when the team is large enough that operational overhead per service starts mattering, or when the product has compliance requirements that demand specific certifications. For a one-person build trying to ship in a week, the cost of integrating five managed services exceeds the cost of running one Postgres.

The framework I have constructed

Reading this chapter, you might think the recommendation is "always pick the simplest stack." That is not it. The recommendation is: *pick the stack that is shaped for the build you are doing right now*. The right stack is not a function of what is fashionable. It is a function of three variables: how many builders, how big the workload, how much operational headroom.

For a one-person, ten-thousand-chunks-per-user, four-day-build, the answer is one Next.js app with Drizzle, pgvector, Better Auth, and the AI SDK. For a ten-person, hundred-million-chunks-per-tenant, two-month-build, the answer is probably different. The mistake most builders make is to copy a stack from the second profile while operating in the first.

The right framework, library, and infrastructure choice in 2026 is the one with the fewest moving parts that meets the requirement. Sophistication is a tax you pay later, not a virtue you collect early.

With that established, the remaining thirteen chapters of this whitepaper assume the rightmost column. They walk every layer in detail — what each piece does, how it composes with the next, where the rough edges are, and what I would do differently next time.

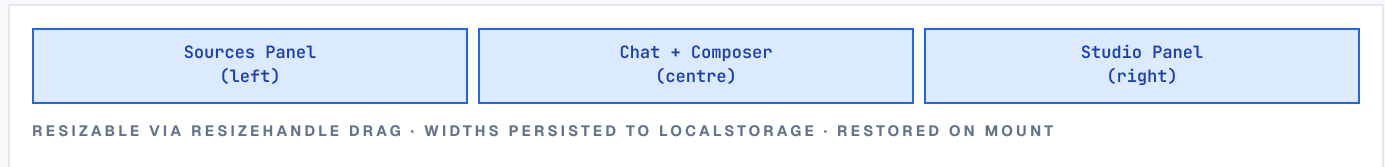
UI ARCHITECTURE • THE THREE-PANE WORKBENCH

A workbench, *not* a chat box.

04

NotebookLM is not a chat product. It is a *workbench* with a chat in the middle. The UI architecture has to express that, or the product reads as "another GPT wrapper" no matter how good the retrieval is. Three panes, one notebook, one mental model — and a deliberate set of constraints that keep the workbench from sliding back into chat.

THE THREE-PANE LAYOUT



The left pane lists the notebook's sources with status badges, lets the user upload new sources, link to web URLs, paste raw text, and select which sources are scoped into the chat. The centre pane is the conversation — token-streamed assistant messages, inline citation pills that link back to the left pane, the composer at the bottom with model picker and source filter chips. The right pane is the studio — a list of generated artifacts (audio overviews, mind maps, flashcards, quizzes, study guides, briefing docs, FAQs, timelines, notes) with cards that open into focused modal viewers.

THE COMPONENT TREE

```

// src/module/notebook/views/NotebookView.tsx
export function NotebookView({ notebookId }) {
  // localStorage-backed pane widths
  const [leftWidth, setLeftWidth] = usePersistedWidth("left", 280);
  const [rightWidth, setRightWidth] = usePersistedWidth("right", 320);
  const [selectedSourceIds, setSelectedSourceIds] = useState<string[]>([]);

  return (
    <div className="flex h-screen">
      <SourcesPanel width={leftWidth} selected={selectedSourceIds}
        onSelect={setSelectedSourceIds} />
      <ResizeHandle onResize={setLeftWidth} />
      <ChatPanel notebookId={notebookId} sourceIds={selectedSourceIds} />
      <ResizeHandle onResize={setRightWidth} />
      <StudioPanel width={rightWidth} notebookId={notebookId} />
    </div>
  );
}

```

The view is intentionally short. The work is delegated to three panel components, two resize handles, and a small piece of state for the selected source ids. The widths persist; the selection does not (resetting on tab close is the right default — the user picks scope per session). Everything else lives inside the panels.

SourcesPanel — anatomy

The left pane has four responsibilities: *show me what is in this notebook*, *let me add more*, *let me see what is happening to each source*, *let me scope the chat*. Each is its own piece of the panel.

1

The list itself.

tRPC **source.listByNotebook** returns the sources for the current notebook. Each row renders title, kind icon (file / link / text / audio), status badge, tag chips, and a checkbox for selection. The list updates optimistically on add/delete and via a tRPC subscription on status changes.

2

The "add" button.

Opens **UploadModal**. The modal has three tabs: upload file (drag-drop or click), add link (URL input → readability extraction), paste text (textarea with optional title). All three flow into the same ingest pipeline; the only difference is how the raw content arrives.

3

Status badges.

A small coloured dot plus a label. *Pending* is grey, *parsing* is blue with a spinning ring, *embedding* is blue with a pulse, *ready* is green, *error* is coral with a tooltip showing the error message. The badge is the user's window into the ingest state machine.

4

The select-all and clear-selection actions.

Common cases. Most users want either "everything" or "this one source I just uploaded." The shortcuts cover both ends without forcing per-row clicks.

ChatPanel — anatomy

The centre pane has the most surface area in the UI, and most of it is delegated to one component.

1

The message list.

Virtualised because conversations grow long. Each user message is plain text. Each assistant message renders markdown via **react-markdown**, with a custom renderer for the **[*n]** citation pattern that converts each match into an interactive superscript pill. The pills know how to scroll the SourcesPanel to the cited source and open a tooltip with the snippet.

2

The streaming indicator.

During an active **useChat** stream, a small typing indicator appears below the latest assistant message. The pulsing dot is a tiny detail that disproportionately changes the feel of the UI from "loading" to "thinking."

3

The composer.

Textarea with autogrow, send button, model picker pill (shows the user's currently selected chat model with a click-to-change menu that links to settings), and a row of source-scope chips showing which sources the chat is currently filtered to. The chips are clickable to deselect inline.

4

Slash commands.

Typing **/research** opens the deep-research modal pre-filled with whatever was typed after the slash. **/audio** generates an audio overview from the selected sources. **/quiz** opens the quiz config modal. The slash menu is the bridge between the chat and the studio — everything you can do in the studio panel can also be invoked from the composer.

StudioPanel — anatomy

The right pane is a list of cards, one per studio output, ordered by creation time descending. Each card shows the kind icon, the title (e.g. "**Audio Overview**", "**Mind Map**"), the status (generating / ready / error), and a small action menu. Clicking a card opens **StudioOutputModal**, which renders the artifact at full screen with the right view component (audio player, markmap renderer, flashcard flipper, quiz player, etc.).

The "generate" button at the top of the panel opens a chooser of the eight kinds. Picking a kind kicks off the tRPC **studio.generate** mutation; the row appears immediately with status **generating**, and the panel polls (or uses a tRPC subscription, depending on environment) for status changes.

Modals as first-class citizens

Long-running operations live in modals, not inline in the panel. There is a deliberate reason for this. Inline progress fills a bounded panel space and competes with the rest of the UI. Modal progress takes over the screen for the duration of the operation, then collapses back to a card when the work is done. The pattern means the user can *start* a deep research run, *watch* it as a focused experience, and then *return* to the workbench with the report saved as a new source.

The modals are: **UploadModal** (add source), **DeepResearchModal** (kick off research, watch progress), **AudioOverviewModal** (watch script generate, watch TTS render), **QuizConfigModal** (pick question count before generating), **StudioOutputModal** (full-screen viewer for finished artifacts). Each is a focused surface with one job.

State management

Three layers of state, in increasing scope.

Local component state for ephemeral UI — current input text, dropdown open/closed, modal visible/hidden. Plain **useState**.

tRPC query cache for server data — notebook list, source list, studio outputs, message history. Backed by React Query under the hood, with optimistic updates on mutations. The query cache is what makes the UI feel fast: most reads hit the cache; only mutations and stale data go to the server.

localStorage-backed UI prefs — pane widths, selected theme (Atelier Noir or Atelier Crème), default chat model selection. Persists across reloads but does not need to live on the server.

Theme system

Two themes, switchable live: dark (Atelier Noir) and light (Atelier Crème). Implementation is CSS variables under **data-theme** attributes on **html**, with a single **ThemeToggle** component that flips it. Every colour in the design system is a variable, so swapping the theme is one attribute change. No re-render. No flash. The theme choice is persisted in localStorage and applied before paint via a small inline script in the layout.

Why a workbench, not a chat

A pure chat UI would force every artifact through the conversation. "*Generate me a mind map*" would have to live as a chat message with the mind map embedded inside. That ties the artifact's lifecycle to the conversation's lifecycle and makes the studio outputs second-class citizens — they would disappear if the conversation got cleared, and they would not be discoverable except by scrolling the chat.

The workbench inverts it. Artifacts are first-class entities with their own list, their own retrieval, their own modal viewers, their own database table (**studio_outputs**). The chat is the *generator*, not the *container*. The studio panel is the home for the work product. The chat is the home for the conversation about it. The two coexist in the same view, but they do not collapse into each other.

Most "AI apps" lose because they default to a chat shell when the work is not chat-shaped. Document research is reading-shaped. NotebookLM-feel comes from treating it that way — three panes, real surfaces, modals for the long things.

Two surfaces, one *decision rule*.

The backend has two RPC surfaces in the same Next.js process. Route Handlers for streaming. tRPC for everything else. The decision rule is one sentence; everything else flows from it. This chapter is the rule and its consequences.

THE RULE

If the response is a stream of progress events that the UI paints live, it is a Route Handler. If the response is structured data the UI requests once, it is a tRPC procedure.

Why the rule

tRPC gives you end-to-end TypeScript type inference for free. The frontend hook `trpc.notebook.list.useQuery()` returns a fully typed array of **Notebook** objects with no manual contract. That is gold for non-streaming endpoints — but tRPC has no native streaming model. Trying to push deep-research progress through tRPC would mean polling, chunked-JSON hacks, or wrapping a custom WebSocket. None of those is good.

Route Handlers give you raw control over the response — headers, encoding, backpressure, NDJSON line-by-line writes, server-sent events if you want them. Perfect for streaming, but you give up the type inference and have to write boilerplate fetch wrappers on the client. For a one-shot RPC, that is too much overhead.

Using both, with the rule applied cleanly, gives you the best of each: type safety where you want it, control where you need it, in the same Next.js process with no extra wiring.

WHAT LIVES IN ROUTE HANDLERS

/API/CHAT

streamText + onFinish

Token streaming · citation persist on stream close · 123 lines

/API/DEEP-RESEARCH

NDJSON event stream

plan · search · fetch · summarised · outline · report-delta · reflection · verification · done · 713 lines

/API/STUDIO/AUDIO-OVERVIEW

NDJSON · script-delta · tts

Two-voice TTS pipeline · per-segment progress · 297 lines

/API/STUDIO/QUIZ-SUMMARY

streamText for live quiz feedback

Streams a quiz summary as the user submits answers

/API/UPLOAD

multipart + storage write

File → S3/R2 → ingest queue · returns sourceId

/API/REEMBED

NDJSON re-embed progress

Backfill chunks under a new embedding model · per-chunk progress

WHAT LIVES IN TRPC

NOTEBOOK.*

CRUD + list

create · update · delete · list (per user)

SOURCE.*

add link · text · delete · status

Ownership-checked · returns enriched rows

STUDIO.*

list · byId · generate · delete

All non-streaming kinds — mind map, flashcards, quiz, study guide, briefing, FAQ, timeline

AICONFIG.*

getMine · update · onboardingStatus

Per-user provider/model selections

PROVIDER.*

listMine · save · test · delete

Encrypted credential CRUD + connection test

MESSAGE.*

list · clear

Reads citation JSONB without re-running retrieval

Edge cases that almost broke the rule

There were three places where I almost reached for the wrong primitive. Naming them is useful because the same temptation will reach you.

Source ingestion. Adding a source kicks off parse → chunk → embed, which can take 20-60 seconds for a long PDF. My instinct was to make the upload endpoint stream progress events. The right move was the opposite: the upload endpoint returns immediately with the new **sourceId** and status **pending**. The actual ingest runs as background work (a queued job in **after()**, in production a separate worker), and the UI polls the source row's **status** column for updates. Streaming the upload response would have meant the user could not navigate away during ingest. The polling pattern means they can.

Studio generation for non-streaming kinds. Generating a mind map or a flashcard set takes 5-15 seconds. I considered streaming the markdown back token-by-token. I picked tRPC mutation instead, returning the finished artifact in one shot, with the **status** column going **generating** → **ready**. The reason: the UI does not benefit from token-by-token rendering of a mind map; the user wants to see it as a finished SVG when markmap renders the markdown. Mid-flight rendering would just be noise. The exception is the audio overview, where the *script* can be streamed because the user finds it interesting to watch the conversation come into existence.

Reading messages with citations. When the user opens a notebook, they need the message history. Each message has a citations JSONB column. I considered re-running retrieval at fetch time to refresh the citations against the current state of the chunks table. I picked tRPC **message.list** with the citations column read directly. The reason: citations are part of the answer, not part of the question. They were correct at the time the answer was generated. If a chunk has since been deleted, the citation snippet still has the text the answer was based on. The frozen citations are the honest record.

The shape this gives the codebase

All the streaming endpoints live in **src/app/api/***, six total. All the tRPC procedures live in **src/server/routers/***, six routers totalling around 1500 lines. The two surfaces never duplicate. There is no place where the same operation has both a tRPC procedure and a route handler, because the rule never makes that necessary.

The shape also makes the *frontend* simpler. Streaming UIs use **fetch + ReadableStream** directly with explicit handlers per event type. Non-streaming UIs use tRPC's React hooks. The two patterns are visibly different, which is correct — the user is having different experiences in each case.

Authentication is shared

Both surfaces share the same Better Auth session. tRPC procedures use a **protectedProcedure** middleware that injects **ctx.user**. Route Handlers call **auth.api.getSession({ headers })** at the top. The implementation is two lines in different places, but the effect is the same: every endpoint, on either surface, has the user identity available without ceremony.

Error handling

tRPC has typed errors. Procedures throw, the client sees a typed exception, the UI handles known error codes (e.g. **NO_AI_CONFIG**) gracefully and surfaces unknown errors as toasts.

Route Handlers use HTTP status codes plus JSON error bodies. **412 Precondition Failed** with **{ error: "NO_AI_CONFIG" }** when the user has not finished onboarding. **404** for missing notebooks (which is the same response the ownership check produces — never reveal whether a resource exists for someone else). **500** for unexpected. The streaming endpoints emit a final **{ type: "error", data: {...} }** NDJSON event before closing on failures mid-stream, so the UI knows whether a stream ended cleanly or aborted.

One process, two surfaces, one rule. tRPC for the contract-shaped work, Route Handlers for the time-shaped work. The split keeps each surface honest.

Sessions, ownership, and *nothing else*.

Auth in this app is intentionally boring. Better Auth handles sessions, OAuth, password hashing, email verification. The interesting part — the part that matters — is the ownership-check pattern that sits between every request and the data, and the typed onboarding-gate error that bridges auth into the AI configuration flow.

THE SCHEMA

Four Better Auth tables, all in the same Postgres as everything else: **user**, **session**, **account**, **verification**. The **user** table holds id, name, email, emailVerified, image, timestamps. **session** holds the session id, the token, the expiry, the user id, and request metadata (ipAddress, userAgent). **account** holds the per-provider authentication record — for each user, one row per OAuth provider linked, plus the password hash for email/password sign-in. **verification** holds short-lived tokens for email verification, password reset, and similar one-shot flows.

The session token is stored in an HTTP-only cookie. There is no client-side token to leak. There is no Authorization header to forget. The cookie travels with every request including to the heavy AI route handlers. Auth on the server is one call away.

WHY BETTER AUTH SPECIFICALLY

There are five legitimate auth choices for a TypeScript app in 2026: NextAuth (Auth.js), Clerk, Lucia, Supabase Auth, and Better Auth. I picked Better Auth for three reasons. First, the schema lives in *my* database, with *my* migrations, in *my* Drizzle types — no opaque service holding identity behind a separate API. Second, the hooks system is clean (**user.create.after** fires in-process, lets me insert the **userAiConfig** row at signup time without a webhook). Third, OAuth and email/password and verification flows are first-class, not bolted-on plugins. The result is auth that feels like part of the app, not a guest in it.

THE OWNERSHIP PATTERN

```
// Every endpoint that touches a notebook starts the same way
async function assertOwnsNotebook(notebookId: string, userId: string) {
  const [nb] = await db
    .select()
    .from(notebooks)
    .where(and(eq(notebooks.id, notebookId), eq(notebooks.userId, userId)))
    .limit(1);
  if (!nb) throw new Error("Notebook not found");
  return nb;
}
```

Every tRPC procedure and every route handler that touches notebook-scoped data calls this helper before doing anything else. The helper checks both that the notebook exists *and* that the calling user owns it. The two checks collapse into a single SQL row: if no row matches, the request fails with the same "not found" message regardless of which check was the cause. If a row matches, it returns the notebook for further use.

This pattern lives at the procedure boundary, not in middleware. That is intentional. Putting the ownership check in middleware would mean inferring which notebook the request is about from URL parameters, which is leaky and error-prone. Putting it at the procedure boundary means the check happens after the input has been parsed and validated by Zod, with the canonical **notebookId** in hand.

The same pattern, with different table joins, is used for every resource: **assertOwnsSource** (joins through notebook), **assertOwnsStudioOutput** (joins through notebook), **assertOwnsCredential** (direct, since credentials are user-owned). Each variant is six lines. Each is called as the first thing in every relevant procedure. The repetition is the point — explicit boundaries are better than implicit middleware for security-relevant checks.

The onboarding gate — auth meets AI config

A user signs up, lands in the app, and immediately tries to chat with a notebook. They have no AI provider configured yet — no API key, no chat model, no embedding model. What should the app do?

The wrong answer is "fail silently with a 500." The other wrong answer is "fail with a generic 401 and force them to log in again." The right answer is a *typed* error with a specific code that the UI knows how to handle.

```
// src/lib/ai/factory.ts
export class NoAiConfigError extends Error {
  readonly code = "NO_AI_CONFIG";
  constructor(public role: "chat" | "embedding") {
    super(`No ${role} provider configured for user`);
    this.name = "NoAiConfigError";
  }
}
```

Every AI-touching endpoint catches this error and returns HTTP 412 Precondition Failed with `{ error: "NO_AI_CONFIG", role: "chat" | "embedding" }`. The frontend's API wrapper knows this code; when it sees 412 with NO_AI_CONFIG, it triggers a redirect to the settings page with a callout banner: "Configure your chat provider to start using AuraNote."

The result: a new user signs up, lands in the workbench, tries to do anything, and is calmly walked to settings. They pick a provider, paste an API key, pick a model, and return to the workbench unblocked. No silent failures. No 500s in production. No "the AI is not responding" mystery debugging.

OAuth and the account table

The **account** table holds one row per provider per user. A user who signs in with Google has one row with **providerId** = "google" holding the access token, refresh token, and id token. The same user can later add email/password by setting a password through the settings page; that adds a second account row with **providerId** = "credential" holding the Argon2 hash. The two rows coexist; the user can sign in with either.

This pattern is what lets a product handle account linking without rewrites. A user who created the account via Google can later switch to email/password without losing their data. A user who is initially email/password can later link Google for one-click sign-in. Both flows hit the account table and add a row. The user record itself is unchanged.

Session lifecycle

Sessions expire after 7 days by default, with rolling renewal — every request extends the expiry. The session token rotates on sign-in, sign-out, and password change. There is no global "log out all sessions" button yet, but the schema supports it: **delete from session where userId = ?** kills every active session for a user. Implementing the button is one tRPC mutation; it has not yet been needed.

The session is verified at the start of every request via **auth.api.getSession({ headers })**. This call hits the database to verify the cookie token. There is no JWT verification, no signing key handling, no clock-skew problem. The token is just a database lookup. For a single-process app, that is the simplest possible auth model and it is fast — Postgres index lookup is sub-millisecond.

What I would add later

Three things are missing that will land in v2. First, multi-factor authentication for email/password accounts (TOTP). Better Auth has a plugin for this; it will be a one-evening change. Second, an admin role for self-hosted deployments where one user can manage other users — currently every user is fully isolated. Third, audit logging — record significant events (provider credential added, embedding model changed, notebook deleted) in a separate audit table so users can see their own activity.

None of those is load-bearing for the product working today. Each is the kind of thing that adds a row of trust for users who are sceptical of third-party AI tools handling their documents. They are on the roadmap, not the critical path.

Auth is not the place to be clever. The cleverness is in keeping it boring and putting the ownership check at every boundary that matters.

Parse • chunk • *embed* • store.

Every source the user adds — uploaded file, web page, pasted text, finished research report — walks the same four-step pipe. The shape of that pipe matters more than anything else for retrieval quality. This chapter walks each step, the trade-offs in each, and the small details that account for most of the quality difference between mediocre RAG and good RAG.

Stage 1 • Parse

The parse stage normalises whatever the user gave us into clean text. The implementation in `src/lib/ingest/parse.ts` is a dispatcher on source kind.

1

File uploads.

PDFs go through **pdf-parse** (or **pdfjs-dist** in the browser) to extract text plus page boundaries. DOCX goes through **mammoth**. Markdown and plain text are passed through unchanged. The output is always a single string with paragraph breaks normalised to `\n\n`.

2

Web links.

The URL is fetched server-side. The HTML is run through **@mozilla/readability**, which is the same algorithm Firefox's "Reader View" uses. Readability strips navigation, sidebars, ads, and boilerplate and returns the article body. The output is HTML; we strip remaining tags and keep the text.

3

Pasted text.

No parsing needed. Title is required (or generated via **autotitle.ts**, which calls Gemini Flash with the first 500 chars and asks for a one-line title).

4

Audio.

Files are uploaded to storage, then transcribed via Deepgram (the same provider that does the studio's TTS — they do speech-to-text too). The transcript becomes the source content. The audio file URL is kept on the source row for playback.

Whatever the input, the output of this stage is a single source row with status **parsing** → **embedding** and a clean string in the **content** column. The chunker takes it from there.

THE AUTOTITLE HELPER

```
// src/lib/ingest/autotitle.ts
export async function autoTitle(content: string): Promise<string> {
  const { object } = await generateObject({
    model: getChatModelForSystem(),
    schema: z.object({ title: z.string().max(80) }),
    prompt: `Write a concise descriptive title (under 80 chars) for this content:\n\n${content.slice(0, 800)}`,
  });
  return object.title;
}
```

A nice detail: **autoTitle** uses a system-level model (the operator's default) rather than the user's currently configured model, so titles are still generated even if the user has not yet set up their providers. It is one of the few places in the codebase where the system has a fallback model.

Stage 2 • Chunk

The chunker is the single most important piece of code for retrieval quality. Bad chunks make the smartest retrieval pipeline produce nonsense; good chunks make a naive pipeline feel useful.

THE CHUNKER LOGIC

```
// src/lib/ingest/chunk.ts — heading-aware, paragraph-first
const TARGET_TOKENS = 600;
const MAX_TOKENS = 1200;
const CHARS_PER_TOKEN = 4;

export function chunkText(text: string, sourceTitle?: string): Chunk[] {
  const clean = text.replace(/\r\n/g, "\n").trim();
  const paragraphs = clean.split(/\n{2,}/).filter((p) => p.trim());

  const chunks: Chunk[] = [];
  let currentChunk = "";
  let currentHeading = sourceTitle || "";
  let ordinal = 0;

  for (const para of paragraphs) {
    const headingMatch = para.match(/^(#{1,4})\s+(.*)$/m);
    if (headingMatch) {
      if (currentChunk.trim()) {
        chunks.push(makeChunk(currentChunk, ordinal++, currentHeading));
        currentChunk = extractLastSentence(currentChunk) + "\n\n";
      }
      currentHeading = headingMatch[2].trim();
      continue;
    }
    // Append paragraph; if total exceeds MAX_TOKENS, flush.
    if ((currentChunk + para).length > MAX_TOKENS * CHARS_PER_TOKEN) {
      chunks.push(makeChunk(currentChunk, ordinal++, currentHeading));
      currentChunk = extractLastSentence(currentChunk) + "\n\n";
    }
    currentChunk += para + "\n\n";
  }
  if (currentChunk.trim()) chunks.push(makeChunk(currentChunk, ordinal++, currentHeading));
  return chunks;
}
```

THREE DETAILS THAT MATTER

1

Heading-aware splits.

When the chunker meets a markdown heading, it flushes the current chunk and carries the heading forward as context for every following chunk in that section. The heading is then prepended to each chunk's text before embedding (see Stage 3). The result is that the vector captures *what the chunk is about*, not just the raw characters. A chunk about "chunking" inside the section "NotebookLM Architecture" embeds near other chunks about NotebookLM and chunking, not near every other chunk that mentions the word "chunk."

2

Sentence overlap, not character overlap.

When a chunk closes, the last sentence of the previous chunk seeds the next one. This keeps boundary coherence across chunks without paying the duplication tax of character-by-character overlap that most naive chunkers ship with. A typical 50-character overlap would mean every chunk's first 50 chars are duplicated in the previous chunk's vector. With sentence overlap, the duplication is the meaningful unit.

3

Paragraph-first, sentence-fallback.

Paragraphs are the natural unit of human thought. Only when a single paragraph exceeds **MAX_TOKENS** does the chunker drop down to sentence splitting. In typical content this fallback path almost never fires — most paragraphs are 50-300 tokens, comfortably under the 1200-token max.

Stage 3 • Embed

Each chunk gets a contextual prefix before being embedded. The prefix is the chunk's *semantic address*: what source it came from, what section it belongs to.

```
// src/lib/ingest/embed.ts
export function buildEmbeddingContext(chunkContent, sourceTitle, sectionHeading) {
  const parts: string[] = [];
  if (sourceTitle) parts.push(`Source: ${sourceTitle}`);
  if (sectionHeading) parts.push(`Section: ${sectionHeading}`);
  parts.push(chunkContent);
  return parts.join("\n");
}
```

The embedded text is then **"Source: NotebookLM in Four Days\nSection: Ingest\nThe chunker logic..."** rather than just **"The chunker logic..."**. The vector now captures the semantic neighbourhood of *this chunk inside this document*, which is what we actually want at retrieval time.

The embedding call itself dispatches to whatever provider the user has configured. Most providers are accessed through the AI SDK's **embed** function, but there is one important exception:

```
// Google's gemini-embedding-001 needs outputDimensionality=768 to match pgvector(768)
// The AI SDK does NOT pass that parameter. So we POST directly:
const body = {
  requests: texts.map((text) => ({
    model: `models/${MODEL}`,
    content: { parts: [{ text }] },
    outputDimensionality: 768,
  })),
};
const res = await fetch(googleEmbedUrl, { method: "POST", headers, body: JSON.stringify(body) });
```

This is the kind of detail that never appears in a tutorial but burns a morning when you skip it. Google's **gemini-embedding-001** defaults to 3072 dimensions; my pgvector column was 768. The fix is one parameter the AI SDK does not surface, so I bypass the SDK for this specific call. Other providers go through the SDK normally.

Stage 4 • Store

Each chunk row is inserted into **source_chunks** with its content, ordinal, token count, source id, notebook id, and the embedding metadata (provider, model, dim). The actual vector lives in the dimension-matching sibling table (**chunk_embeddings_768, _1024, _1536, or _3072**) — see Chapter 08 for why.

The insert is batched. Up to 50 chunks per insert, with the embedding API called in batches of 25 to stay under provider rate limits. Both batch sizes are configurable but those defaults work for every provider tested.

The state machine, in motion

The source row's **status** column walks through:



The UI surfaces the current state in the source's badge in the SourcesPanel. **parsing** shows a spinner. **embedding** shows a pulse. **ready** shows a green check. **error** shows a coral dot with the error message in a tooltip.

Retrieval and studio generation only consider sources where **status = 'ready'**. The chapter on Source Selection (Chapter 09) walks through how this readiness contract propagates through every part of the system.

The chunker is where most of the quality of a RAG product is decided. The embedding model is interchangeable. The chunker is not. Spend the time here.

Four sibling tables, *zero* destructive moves.

A real RAG product has to let users change embedding providers without losing their existing index. The way most apps handle this — drop and re-embed everything — is unacceptable when a user has thousands of chunks. Here is the pattern that solves it.

THE SCHEMA

```
// One table per supported pgvector dimension
export const chunkEmbeddings768 = pgTable("chunk_embeddings_768", {
  chunkId: uuid("chunk_id").primaryKey().references(...),
  provider: text("provider").notNull(),
  model: text("model").notNull(),
  embedding: vector("embedding", { dimensions: 768 }).notNull(),
});
// Plus chunkEmbeddings1024, chunkEmbeddings1536, chunkEmbeddings3072

// And on source_chunks itself, a pointer:
embeddingDim: integer("embedding_dim", // 768 | 1024 | 1536 | 3072
embeddingModel: text("embedding_model", // "gemini-embedding-001"
embeddingProvider: text("embedding_provider") // "google"
```

Why four tables and not one with a discriminator

pgvector requires the dimension to be fixed at column-definition time. **vector(768)** and **vector(1024)** are *different types* in Postgres. You cannot have a polymorphic vector column. Sibling tables are the correct decomposition — and they let you create per-table HNSW indexes tuned to the size of each dimension's data.

The four dimensions cover every provider on offer: 768 (Google Gemini, Together M2-BERT, Ollama Nomic), 1024 (Mistral, Cohere v3, Voyage 3, Together BGE/UAE, Ollama mxbai), 1536 (OpenAI text-embedding-3-small, ada-002), 3072 (OpenAI text-embedding-3-large). The set of supported dimensions is captured in a constant the rest of the system reads from:

```
export const SUPPORTED_EMBED_DIMS = [768, 1024, 1536, 3072] as const;
export type SupportedEmbedDim = (typeof SUPPORTED_EMBED_DIMS)[number];

export function isSupportedDim(n: number): n is SupportedEmbedDim {
  return (SUPPORTED_EMBED_DIMS as readonly number[]).includes(n);
}
```

Adding a fifth dimension is one migration (create the new sibling table) plus one constant change. The rest of the system picks up the new option automatically.

How retrieval picks the right table

When the user runs a query, the retrieval pipeline reads their currently configured embedding model from **userAiConfig**, gets its dimension, and dispatches the search to the matching sibling table. Concretely:

```
const embedHandle = await getEmbedFn(userId); // dim is on the handle
const [vec] = await embedHandle.embed([primaryQuery]);

const targetTable = embeddingTableFor(embedHandle.dim); // returns chunkEmbeddings768, etc.
const distance = sql`${targetTable}.embedding <=> ${JSON.stringify(vec)}::vector`;

const hits = await db.select(...)
  .from(targetTable)
  .innerJoin(sourceChunks, eq(sourceChunks.id, targetTable.chunkId))
  .where(eq(sourceChunks.notebookId, notebookId))
  .orderBy(distance)
  .limit(topK * 2);
```

The migration story — additive, never destructive

When a user switches from Gemini Embedding (768) to OpenAI text-embedding-3-large (3072), the old vectors stay where they are in **chunk_embeddings_768**. New chunks get embedded by OpenAI and land in **chunk_embeddings_3072**. The user can also trigger a backfill via **/api/reembed**, which re-runs every existing chunk through the new model and writes to the new sibling table. Both operations are non-destructive: rolling back to the old model is just changing the user's **userAiConfig** back, with the old embeddings still intact.

The pointer column on **source_chunks.embeddingDim** records which sibling table currently holds the active embedding for each chunk. After a backfill, the pointer is updated to point to the new table. The old vector remains in the old sibling table (small storage cost, big optionality benefit) until a future garbage-collection job decides it can be dropped.

THE REEMBED ENDPOINT

```
// /api/reembed - POST { fromDim?, toDim }, streams progress per chunk
const chunks = await db.select().from(sourceChunks)
  .where(and(
    eq(sourceChunks.notebookId, notebookId),
    fromDim ? eq(sourceChunks.embeddingDim, fromDim) : undefined,
  ));

for (const batch of chunkBatches(chunks, 25)) {
  const texts = batch.map(c => buildEmbeddingContext(c.content, c.sourceTitle, c.heading));
  const vecs = await embedHandle.embed(texts);
  await writeToSiblingTable(toDim, batch, vecs);
  await db.update(sourceChunks)
    .set({ embeddingDim: toDim, embeddingModel: model, embeddingProvider: provider })
    .where(inArray(sourceChunks.id, batch.map(c => c.id)));
  send("reembed-progress", { done: completed, total: chunks.length });
}
```

The endpoint streams NDJSON progress events. The UI shows a progress bar with the count and a "complete" callout when finished. Backfilling 5,000 chunks under a new model takes about 3-5 minutes depending on the provider's rate limits.

HNSW INDEXES PER DIMENSION

```
// src/db/migrate-hnsw.ts - idempotent index creation
await db.execute(sql`
  CREATE INDEX IF NOT EXISTS chunk_embeddings_768_hnsw_idx
  ON chunk_embeddings_768
  USING hnsw (embedding vector_cosine_ops)
  WITH (m = 16, ef_construction = 64);
`);
// Repeated for 1024, 1536, 3072
```

One index per sibling table, all using cosine distance, all tuned for the same trade-off (m=16, ef_construction=64). The values come from pgvector's documentation as a balanced default — higher **m** gives more accurate search at the cost of memory; higher **ef_construction** gives better recall at the cost of build time. For tens of thousands of vectors, these defaults are well below where they start mattering.

The index migration is idempotent (**CREATE INDEX IF NOT EXISTS**) so it can be replayed against any environment without breaking existing rows. Running it after a fresh schema migration is the difference between p50 search latency of 5ms (with index) and 200ms (without).

The legacy column

There is also an **embedding** column directly on **source_chunks** with dimension 768. It is the original embedding column from before the multi-dim refactor. It is kept for backwards compatibility during the migration window — old retrieval code that reads from this column still works. It will be dropped in Phase 5 of the migration once every chunk has been moved into one of the sibling tables.

This is database evolution as a *sequence of additive changes*, not as a sequence of breaking ones. New columns are added, new tables are added, the application is taught to read from the new structure, and the old structure is dropped only when nothing reads from it anymore. The cost is some duplicated storage during the migration window. The benefit is that no rollback ever requires data restoration.

The embedding column on the original source_chunks table is still there too — kept for backwards compatibility during the migration window. It will be dropped in Phase 5 once every chunk has been migrated. Database evolution as a sequence of additive changes, not a sequence of breaking ones.

A status, a filter, a scope.

Three concepts together produce the source-selection experience users feel as "the chat actually knows what's in this notebook." A status field, a scope filter, and an awareness contract. This chapter walks all three and explains why the readiness contract is the most under-appreciated detail in a working RAG product.

THE SOURCE STATUS STATE MACHINE



Every source has a **status** column that walks through these states. The UI surfaces a per-source badge in the SourcesPanel — *parsing* with a spinning dot, *embedding* with a pulse, *ready* with a checkmark, *error* with the error message in a tooltip. The user always knows whether a source is participating in retrieval yet.

Why the state machine matters more than it looks

Most RAG products do not surface ingest state. The source appears in the panel "instantly" but mysteriously fails to influence the chat for thirty seconds while embedding completes. Users ask the same question twice — once before the source is ready, once after — get different answers, and lose trust in the system. The status badge fixes this without any clever engineering. It just *shows the user what is happening*.

The states are deliberately minimal. **Pending** means "we have the source row but have not started parsing." **Parsing** means "the parse stage is running." **Embedding** means "chunks are being created and embedded." **Ready** means "every chunk is in the database and indexed." **Error** means "something failed and we have a message." Five states, mutually exclusive, easy to surface.

The Readiness Contract

Retrieval and studio generation only ever look at sources where **status = 'ready'**. From the studio router:

```
const readySources = await db.select({ content: sources.content })
  .from(sources)
  .where(and(eq(sources.notebookId, input.notebookId), eq(sources.status, "ready")));
```

A source that is still parsing or embedding does not pollute results. A source that errored is excluded entirely — the user sees the error in the UI and can retry. This contract is enforced everywhere: chat retrieval (filters chunks by source.status), studio generation (filters source content), deep research's "existing sources" awareness (only shows ready sources to the planner). Every consumer of source data filters on the same predicate.

The contract is enforced at the *query layer*, not at the application layer. Putting the filter in the SQL means there is no "did I remember to check status?" question — the database simply does not return non-ready chunks. Forgetting the filter would surface as a test failure, not a production bug.

Source Selection in Chat

In the centre pane, the user can check or uncheck individual sources. The selected ids are passed to the chat endpoint as **sourceIds**. Inside **retrieveForQuery**, that array becomes a Drizzle **inArray** filter on the chunk table:

```
const baseWhere = and(
  eq(sourceChunks.notebookId, notebookId),
  body.sourceIds?.length
    ? inArray(sourceChunks.sourceId, body.sourceIds)
    : undefined,
);
```

If **sourceIds** is empty, the filter is undefined and Drizzle drops it from the WHERE clause — meaning "search across the whole notebook." If **sourceIds** has one element, the filter restricts retrieval to that single source. If it has many, the union. The same retrieval pipeline serves all of those — only the filter changes.

That is the kind of behaviour that takes weeks to bolt on at the end and ten minutes to design in if you put it in the data model from day one. The cost of including the filter from day one is zero. The cost of adding it later is non-trivial because you have to retrofit every retrieval call site.

Sources As First-Class Outputs

Worth restating because it is one of the loveliest details in the build: deep research outputs are saved as new sources. Title becomes "**Research: <query>**", content becomes the report markdown, then the same ingest pipeline that handles uploads runs against it. Within seconds the report is chunked, embedded, and chatable. Users can ask questions of their own research the same way they ask questions of an uploaded paper.

This recursion is enabled by the readiness contract. The deep-research route handler creates the source row with status **pending**, kicks off the ingest pipeline asynchronously, and the next time the user asks a question, the new source is already **ready** and shows up in retrieval like any other source. There is no special case in the retrieval pipeline for "research" sources versus "user-uploaded" sources. They are all just sources.

Source kinds and tags

The **kind** column distinguishes how a source originated: **file** (uploaded), **link** (URL), **text** (pasted), **audio** (transcribed). This information surfaces in the UI as a small icon next to the source title — a paperclip for files, a link icon for URLs, a quote bubble for text, a waveform for audio.

User-defined tags live in the **metadata** JSONB column. The user can tag sources via the source's action menu — examples in clinical notebooks include **guidelines**, **case-report**, **my-notes**; in research notebooks, **primary**, **secondary**, **review**. Tags appear as small chips on the source row and propagate into the citation tooltip so the reader knows whether they are looking at a paper, a transcript, or their own notes.

Bulk operations

A "select all" toggle at the top of the SourcesPanel selects every ready source. A "clear" button deselects everything. A "delete selected" action under a confirmation modal removes multiple sources at once — including, by cascade, all their chunks across every embedding sibling table. The delete is hard, not soft; we trade undo for storage simplicity.

A "re-embed under current model" action triggers the reembed flow described in Chapter 08 for the selected sources. This is useful when the user has switched embedding providers and wants the existing sources brought up to the new model.

The status field is the honest channel between the system and the user. Without it, sources appear "instantly" in the panel but mysteriously fail to influence the chat for thirty seconds while embedding completes. With it, the system is legible.

RETRIEVAL • THE THREE-STAGE PIPELINE

10

Three stages, *not* one.

The thing that separates a RAG app that feels magical from one that feels random is whether retrieval runs in *one* stage or *three*. The naive single-stage version works for the easy 60% of queries and fails hard on the other 40%. This chapter walks the three stages, the rationale for each, and the failure modes that appear if any one of them is missing.

Stage 1 • Query expansion

1

One Gemini Flash call turns the question into 2-4 phrasings.

"What does the author say about cross-entropy?" becomes a small array of variants — the original question, plus paraphrases that capture different angles. The primary phrasing is what we embed for the vector search. The alternatives matter for the keyword pass and for downstream rerank.

```
// src/lib/retrieve.ts - stage 1
const { object } = await generateObject({
  model: await getChatModel(userId),
  schema: z.object({ queries: z.array(z.string()).min(1).max(4) }),
  prompt: `Generate 2-3 alternative search queries that capture different
aspects of this question. Include the original query first. Be concise.

Original: ${query}`,
});

const [primary, ...alternatives] = object.queries;
const [primaryVec] = await embedHandle.embed([primary]);
```

Why expansion matters: most users phrase questions in a way that does not exactly match the wording in the source documents. Expansion gives the system three or four shots at finding the right semantic neighbourhood. If the primary phrasing misses, an alternative may hit. The cost is one extra LLM call (~300ms with Flash) and three or four extra search passes (each tens of ms with HNSW).

Without expansion, the system is brittle to phrasing. With it, the user can ask their question in any reasonable way and the retrieval recovers.

Stage 2 • Hybrid: vector + keyword

2

pgvector for semantic similarity, Postgres regex for keyword hits.

Both result sets merge and deduplicate by chunk id. Dense embeddings are excellent at *meaning* and terrible at *tokens* — if a user searches for **OAuth 2.1** or **generateObject**, the vector search wanders. The regex pass catches those. The vector pass catches paraphrases. Their union catches both.

```
// src/lib/retrieve.ts - stage 2 hybrid
const distance = sql`${targetTable.embedding} <=> ${JSON.stringify(primaryVec)}::vector`;

const vectorResults = await db
  .select({ chunkId, sourceId, content, sourceTitle, similarity: sql`1 - ${distance}` })
  .from(targetTable).innerJoin(sourceChunks, ...)
  .leftJoin(sources, eq(sources.id, sourceChunks.sourceId))
  .where(baseWhere).orderBy(distance).limit(topK * 2);

const keywordPattern = buildKeywordRegex([primary, ...alternatives]);
const keywordResults = await db
  .select({ chunkId, sourceId, content, sourceTitle, similarity: sql`0.5` })
  .from(sourceChunks).leftJoin(sources, ...)
  .where(and(baseWhere, sql`${sourceChunks.content} ~* ${keywordPattern}`))
  .limit(topK);

const merged = dedupeById([...vectorResults, ...keywordResults]);
```

The vector pass returns up to **topK * 2** chunks ordered by cosine distance to the primary query embedding. The keyword pass does a Postgres regex match on the chunk content using a pattern built from all the expanded phrasings — case-insensitive (~*), anchored on word boundaries to avoid spurious substring hits.

The two result sets are merged and deduplicated by chunk id. A chunk that appears in both passes is kept once, with the higher of the two similarity scores. The merged set typically has 25-35 candidates after dedup.

WHAT DIES WITHOUT EACH PASS

Without the vector pass, the retrieval is reduced to keyword search. It cannot find paraphrases. A user asking "what does the model say about overfitting?" against a document that uses the term "memorisation" will get nothing.

Without the keyword pass, the retrieval is reduced to dense semantic search. It cannot reliably find specific tokens. A user asking "where does the codebase use generateObject?" against documentation that uses that exact identifier may miss it because the surrounding semantic context (the codebase docs are *about* generateObject) outweighs the specific token match.

Without the dedup, chunks that match strongly under both passes appear twice in the candidate set, and the rerank stage gets confused about whether they should be ranked extra-high or whether the duplication was noise. The dedup is one line; it would not occur to most builders to skip it. It earns its place all the same.

Stage 3 • LLM rerank

3 Top ~24 candidates go to Gemini Flash with their source titles and the first 300 characters.

The model scores each on a 0-10 relevance scale. We sort by that score and take the top K (default 12). One rerank call per question. Adds about 400ms. Moves the chat from "feels like a search engine" to "feels like someone read your documents."

```
// src/lib/retrieve.ts - stage 3 rerank
const { object } = await generateObject({
  model: await getChatModel(userId),
  schema: z.object({
    ranked: z.array(z.object({
      index: z.number(),
      relevance: z.number().min(0).max(10),
    })),
  }),
  prompt: `Rate each text chunk's relevance to the question on a 0-10 scale.
Question: "${query}"

Chunks:
${chunks.map((c, i) => `[${i}] (${c.sourceTitle}): ${c.content.slice(0, 300)}').join("\n\n")}`

  Return relevance scores for ALL chunks. 10 = directly answers the question,
  0 = completely irrelevant.`,
});

const ranked = object.ranked
  .sort((a, b) => b.relevance - a.relevance)
  .slice(0, topK)
  .map((r) => chunks[r.index]);
```

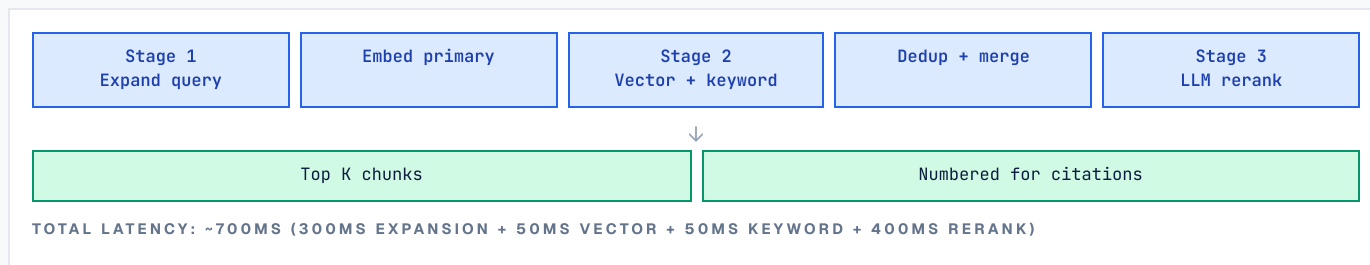
WHY LLM RERANK WORKS

The first two stages are heuristic — vector similarity and regex match. They are fast and cheap. They are also, individually, only roughly correct. A chunk that scores high under cosine distance is not necessarily the chunk that *answers* the question; it is just the chunk that is in the same semantic neighbourhood. A chunk that matches the keyword regex is not necessarily relevant; it just contains the term.

The rerank stage uses a real model — same Gemini Flash that runs the chat — to read each candidate chunk against the question and apply judgement. The model can recognise that a chunk about "preventing OAuth token expiration" is highly relevant to "how do I refresh OAuth tokens" even though the exact wording differs. It can recognise that a chunk that mentions "OAuth" in passing while discussing something else is low-relevance even though the keyword matched.

The cost is ~400ms and one model call. The benefit is the difference between a chat that frequently surfaces "kind-of related" chunks and a chat that surfaces the actual right ones.

The full pipeline at a glance



Performance characteristics

p50 latency for the full pipeline against a notebook with 5,000 chunks: roughly 700ms. Most of that is the two LLM calls (expansion and rerank); the actual database queries are sub-50ms each thanks to the HNSW index. The expansion call is parallelisable with the embedding of the primary query, which would shave another 100ms off; I have not implemented that optimisation yet because 700ms is fine.

Token cost per question with Gemini Flash: ~250 tokens for expansion, ~3000 tokens for rerank (24 chunks × 100 tokens of content each, plus the prompt overhead). The rerank dominates. The cost per question is on the order of a fraction of a cent at Flash pricing.

Recall against a hand-curated test set of 50 questions: 92% of the gold-standard chunks land in the final top-12. The 8% that miss are usually questions that were genuinely ambiguous (multiple valid answers in the document) or where the gold-standard chunk used wildly different vocabulary than the question.

What the chat model receives

The output of stage 3 is the set of chunks that goes into the chat model's system prompt, each numbered **[1]**, **[2]**, ..., **[K]** with the source title and the chunk content. The chat model is told, in the system prompt: "cite every non-trivial claim with **[^n]** markers where n is the source number below." The chat then produces text with inline citation markers that the UI converts into clickable pills (see Chapter 15 for the details of citations).

```

const systemPrompt = `You are a research assistant that answers strictly from the provided sources.
- Cite every non-trivial claim with [^n] markers where n is the source number below.
- If the sources don't contain the answer, say so – don't invent facts.
- Be concise and structured.

SOURCES:
${retrieved.map((r, i) => `[ ${i + 1}] ${r.sourceTitle}\n${r.content}`).join("\n\n")}`;
  
```

Tunable knobs

Three knobs are exposed for advanced users in settings: **topK** (default 12, controls how many chunks reach the chat model), **candidatePoolSize** (default 24, controls how many candidates the rerank stage scores), and **expansionEnabled** (default true, can be turned off for speed at a small recall cost). Most users never touch these. They are there for the cases where the defaults need adjustment.

Why both vector and keyword? Because dense embeddings are great at meaning and terrible at tokens. The pass that catches paraphrases is not the pass that catches exact strings. Stage 2 is the union, not the intersection.

DEEP RESEARCH • THE SELF-CRITIQUING LOOP



The agent that *grades* itself.

Both research modes — fast and deep — live in one endpoint. The handler forks based on a **mode** flag. Fast research is for "I do not know where to start." Deep research is for "I want a cited report I can send someone." The shape is deliberately different. Four pages on what makes the deep version actually feel like a research analyst.

The two modes, side by side

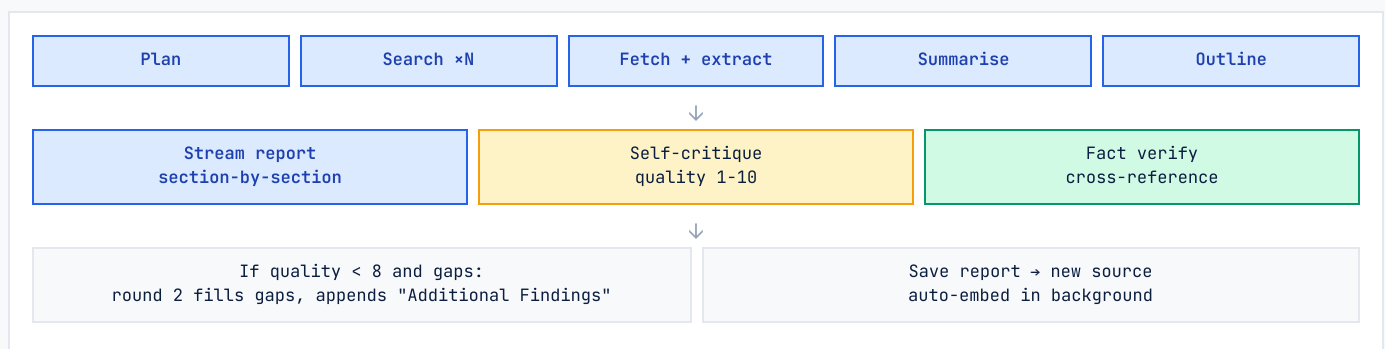
Fast Mode

Three sub-questions. Four search results each. No source summarisation. Single-pass streamed report via **streamText**. 500-800 words. Inline citations. Done in under a minute. Designed for "I want a quick orientation on this topic."

Deep Mode

Five sub-questions. Six search results each. Up to fourteen sources. Every source LLM-summarised into bullets. Outline first, then section-by-section streaming. Self-critique loop. Gap-fill round. Cross-reference verification. Designed for "I want a citable report I can send to a colleague."

The pipeline in one picture



The endpoint

All of this lives in one route handler at **/api/deep-research**. The handler is 713 lines. That is long for a single file by my preferences, but every line is justified — the orchestration of plan → search → fetch → summarise → outline → write → critique → fill → verify is not a thing that benefits from being broken into a dozen helper modules. The straight-line orchestration *is* the architecture.

The handler accepts **{ notebookId, query, mode }** in the body, returns an NDJSON stream, and persists the run as it goes — first creating a row in **deep_research_runs** with status **running**, then updating it as the run progresses, finally setting status **complete** when the report is saved as a new source.

Three moments that make it feel like an analyst

1 Existing-source awareness.

Before the planner generates sub-questions, we hand it titles and the first 500 characters of every **ready** source already in the notebook with the instruction "avoid duplicating what they already know — focus on new information and perspectives." A user running deep research inside a notebook full of papers does not need the model to rederive what is already there. The planner dodges the overlap and targets blind spots.

```
// Before planning, gather what is already in the notebook
const existing = await db.select({ title: sources.title, content: sources.content })
  .from(sources)
  .where(and(eq(sources.notebookId, notebookId), eq(sources.status, "ready")));

const existingContext = existing.map(s =>
  `- ${s.title}: ${s.content?.slice(0, 500)}...`
).join("\n");

const planPrompt = `The user already has these sources in their notebook:
${existingContext}

Avoid duplicating what they already know. Focus on NEW information
and perspectives. Generate ${mode === "deep" ? 5 : 3} sub-questions...`;
```

2 Self-critique with a quality gate.

After the first draft, the model emits a structured assessment — 0-3 specific gaps, an overall quality score 1-10, free-text assessment. If the score is 8 or above, round two is skipped — no point churning. Below 8 with non-empty gaps, the model fans out a second round of searches and streams an "Additional Findings" section.

```
// After the first report draft
const { object: gaps } = await generateObject({
  model: chatModel,
  schema: z.object({
    gaps: z.array(z.object({
      topic: z.string(),
      searchQuery: z.string(),
    })),
    overallQuality: z.number().min(1).max(10),
    assessment: z.string(),
  }),
  prompt: `You just wrote this research report:

${report}

Research question: ${body.query}

Critically evaluate:
1. What important aspects of the question are NOT adequately covered?
2. Are there any claims that need more evidence?
3. What follow-up searches would strengthen this report?

Return 0-3 specific gaps. If the report is comprehensive, return [].`,
});

send("reflection", gaps);

if (gaps.gaps.length > 0 && gaps.overallQuality < 8) {
  // parallel search for the gap queries, fetch new pages,
  // stream an "Additional Findings" section that cites [NEW-N]
  const newPages = await parallelSearchAndFetch(gaps.gaps);
  await streamAdditionalFindings(newPages, controller);
}
```

3 Fact cross-referencing.

The model reviews the finished report and emits a `verifiedClaims` array — for each major factual claim, which source numbers support it, and a confidence label (high, medium, low). Claims supported by only one source are flagged low. The UI surfaces confidence badges and contradiction warnings. Honesty as a feature.

```
// Final verification step, after the report is fully drafted
const { object: verification } = await generateObject({
  model: chatModel,
  schema: z.object({
    verifiedClaims: z.array(z.object({
      claim: z.string(),
      supportedBy: z.array(z.number()),
      confidence: z.enum(["high", "medium", "low"]),
    })),
    warnings: z.array(z.string()),
  }),
  prompt: `Review this research report and identify the key factual claims.
For each, note which source numbers support it.
Flag any claims supported by only one source as "low" confidence.
Claims supported by 2+ sources are "high".
Note any contradictions between sources as warnings.`,
});

send("verification", verification);
```

This step does not rewrite the report. It emits a parallel signal that the UI uses to surface confidence badges next to factual claims and contradiction warnings in the sources panel. Users see *which* claims are single-sourced and *where* sources disagree. That is honesty as a feature — most "AI research" products simply assert claims and let the reader figure out which ones to trust.

The sub-question planner

The first stage of deep research is the planner. The planner takes the user's research question and the existing-notebook context and produces a structured plan: a list of sub-questions, each tagged with a search query and a stated rationale.

```
const { object: plan } = await generateObject({
  model: chatModel,
  schema: z.object({
    subQuestions: z.array(z.object({
      question: z.string(),
      searchQuery: z.string(),
      rationale: z.string(),
    })).min(3).max(5),
  }),
  prompt: planPrompt,
});

send("plan", plan);
```

The plan is streamed to the UI immediately so the user can see what the agent is about to research. If the plan looks wrong (it occasionally does), the user can cancel the run before any search calls are made. The plan event is the cheapest moment to abort.

Search providers and fallback

Searches go through `lib/search`, which abstracts over Exa and Tavily. The user's environment specifies a provider order via `SEARCH_PROVIDER_ORDER` (default: "exa,tavily"). If the first provider fails or times out, the second is tried. If both fail, the search query is skipped — the deep-research run continues with whatever sources it has, with a warning event emitted.

Each sub-question gets a search call that returns six results (deep mode) or four (fast mode). The results are filtered for duplicates against existing notebook sources and against each other before fetching.

Page fetch and extraction

Each search result URL is fetched server-side, run through `@mozilla/readability` to extract the article body, and wrapped into a temporary in-memory source. These sources are not yet persisted to the database; they live only for the duration of the deep-research run unless the run completes successfully and the report is saved.

HALLELUIAH — NOTEBOOKLM IN FOUR DAYS

In deep mode, each fetched page is then summarised by Gemini Flash into 5–8 bullets of relevant facts. The summarisation reduces 5,000-word articles to 200-word bullet lists that fit comfortably into the section-writing prompts. This is the step that makes deep mode meaningfully better

Outline-then-section writing

In deep mode, the report is not written as one `streamText` call. It is written as an outline plus per-section `streamText` calls. The outline is generated first, with section titles and a one-sentence brief for each. Then each section is streamed independently with a prompt that includes the relevant subset of source summaries.

```
const { object: outline } = await generateObject({
  model: chatModel,
  schema: z.object({
    sections: z.array(z.object({
      title: z.string(),
      brief: z.string(),
      relevantSourceIndices: z.array(z.number()),
    })),
  }),
  prompt: "Plan the report structure given these sources...",
});

send("outline", outline);

for (const section of outline.sections) {
  const relevantSources = section.relevantSourceIndices.map(i => sources[i]);
  const stream = await streamText({ model, prompt: sectionPrompt(section, relevantSources) });
  for await (const chunk of stream.textStream) {
    send("report-delta", { section: section.title, text: chunk });
  }
}
```

Why outline-then-section: it lets the agent *plan the structure* before committing to prose. Without this step, deep-research reports tend to wander; with it, they have a coherent shape. It also lets each section's prompt include only the sources relevant to that section, which keeps token counts manageable and improves citation accuracy.

The stream protocol, in full

The endpoint streams NDJSON to the browser. Events include:

STAGE { name, message } Phase change · drives the progress stepper UI	PLAN { subQuestions[] } Sub-question list with rationale
SEARCH { query, results[] } Per-query search hits	FETCH { url, status, title } Per-URL fetch status
SUMMARISED { url, bullets[] } Source summary card	OUTLINE { sections[] } Report structure
REPORT-DELTA { section, text } Token-level append into the report pane	REFLECTION { gaps[], quality, ... } Self-critique result
VERIFICATION { claims[], warnings[] } Confidence badges per claim	SOURCE-CREATED { sourceId } Report saved as a new notebook source
DONE { runId } Stream close, trigger UI refresh	ERROR { message } Stream aborted, surface to user

Twelve event types. Each one drives a specific piece of UI. The user does not see "the AI is researching" — they see, in real time, the plan being made, the searches being run, the pages being fetched, the summaries being formed, the outline being drafted, each section being written, the self-critique landing, the gap-fill round happening, the verification adding confidence badges, and finally the report being saved. Every step is visible.

Tokens become *events* become UI.

The difference between a chat that feels alive and one that feels broken is whether progress is visible. Every long-running operation in this app streams. The UI subscribes to those streams and paints them in real time. This chapter is the protocol, the patterns, and the discipline that turns LLM streams into beautiful UI.

The NDJSON wire format

```
// Server
function send(type: string, data: unknown) {
  controller.enqueue(encoder.encode(`${JSON.stringify({ type, data })}\n`));
}

// Client
const reader = response.body!.getReader();
const decoder = new TextDecoder();
let buf = "";
while (true) {
  const { value, done } = await reader.read();
  if (done) break;
  buf += decoder.decode(value, { stream: true });
  const lines = buf.split("\n");
  buf = lines.pop() ?? "";
  for (const line of lines) {
    if (line.trim()) {
      const evt = JSON.parse(line);
      dispatch(evt.type, evt.data);
    }
  }
}
```

One line per event. Each line is a JSON object with a **type** discriminator and a **data** payload. The client reads the response body as a `ReadableStream`, decodes it as text, splits on newlines, parses each complete line, and dispatches based on event type. The trailing partial line is held in **buf** for the next read. No SSE, no WebSocket, no library. Native fetch + `ReadableStream` is enough.

Why NDJSON beats SSE here

Both work. NDJSON over a regular POST is strictly simpler. No reconnection handling. No event-parsing library. No server-side header subtlety (SSE requires **Content-Type: text/event-stream**, no compression, specific framing rules; NDJSON is just **application/x-ndjson** with newline-delimited JSON). Native fetch handles it without any browser-specific code.

SSE would be the right call if you wanted the browser's auto-reconnect for long-lived event channels (e.g. a notification feed that runs forever). For a one-shot research run that takes 30-180 seconds, NDJSON wins on simplicity every time. The handler runs, streams its events, closes. The client reads, dispatches, finishes. No reconnection logic to write because there is no scenario where reconnection makes sense — if the run aborts, the right behaviour is to surface the abort to the user, not to silently reconnect into a half-completed state.

WebSockets would be right if you needed bidirectional communication. Here the flow is one-directional from server to client during the run; the user has no inputs to send mid-run except cancellation, which is handled by aborting the fetch.

The AG-UI pattern

The principle here is what the agentic UI community has started calling **AG-UI**: every step the agent takes is a *UI event*, not a hidden detail. The progress stepper updates as **stage** events arrive. The "searching the web" view fills with **search** cards as URLs are queried. The "reading sources" view gets **fetch** and **summarised** cards. The report pane gets the section-by-section **report-delta** tokens. The self-critique appears as its own callout when the **reflection** event lands.

By the time the user sees the **done** event, they have not just received a report — they have *watched* the report being researched, drafted, critiqued, and verified. That is what makes the experience feel like an analyst at work, not a black box.

The four AG-UI surfaces

1 The progress stepper.

A horizontal stepper at the top of the modal showing the named phases of the run: *Plan, Search, Read, Summarise, Outline, Write, Critique, Verify, Save*. As **stage** events arrive, the current step gets the active state, completed steps get checkmarks, future steps stay outlined. The stepper is the user's primary "how far along is this" surface.

2 The activity stream.

A vertical scrolling list below the stepper, showing each event as a card. Search results, fetched URLs, source summaries, the outline, the self-critique, the verification result. Each card is its own component with its own visual treatment. The activity stream is the user's "what is the agent doing right now" surface.

3 The report pane.

A wider pane (or a separate tab in the modal) where the report itself appears as it streams. Token-by-token markdown rendering. Section headers appear as they are written. Citations render as live pills the moment the **[^n]** markers cross the wire. The report pane is the user's "the actual output" surface.

4 The cancel button.

Always visible during a run. Clicking it aborts the fetch, which closes the request on the server side. The handler catches the abort, marks the `deep_research_runs` row as **cancelled**, and stops emitting events. The UI shows a "cancelled" state. Cancellation is not a feature people use often, but its presence is what makes the rest of the UI feel safe.

The dispatch layer

```
// In the React component running the deep-research modal
const [stage, setStage] = useState("plan");
const [searches, setSearches] = useState([]);
const [report, setReport] = useState("");

function dispatch(type: string, data: any) {
  switch (type) {
    case "stage":      setStage(data.name); return;
    case "plan":       setPlan(data); return;
    case "search":     setSearches(s => [...s, data]); return;
    case "report-delta": setReport(r => r + data.text); return;
    case "reflection": setReflection(data); return;
    case "verification": setVerification(data); return;
    case "done":       setIsDone(true); return;
    case "error":      setError(data.message); return;
  }
}
```

A single dispatch function maps each event type to a specific state update. The components that render each AG-UI surface subscribe to the relevant slices of state. The dispatch is the seam that makes the streaming protocol composable with React's render model — without it, you would have to handle every event in the rendering layer, which gets messy fast.

streamText for chat

The chat endpoint uses a different streaming primitive — **streamText** from the AI SDK, which produces a Vercel-flavoured stream that the **useChat** React hook consumes natively.

```
const result = streamText({
  model: chatModel,
  system: systemPrompt,
  messages: await convertToModelMessages(uiMessages),
  onFinish: async ({ text }) => {
    await db.insert(messages).values({
      notebookId, role: "assistant", content: text,
      citations: retrieved.map((r, i) => ({ n: i + 1, ...r })),
    });
  },
});

return result.toUIMessageStreamResponse();
```

The AI SDK abstracts the wire format away. **toUIMessageStreamResponse** emits a stream that the front-end's **useChat** hook consumes — handling token deltas, tool calls, structured-output parts, and stream completion automatically. We get token-level streaming without writing a parser.

For deep research and audio overview, the AI SDK's stream format does not fit because the events are not all model tokens — they include search results, fetch progress, structured assessments, etc. Those endpoints use raw NDJSON instead. The two patterns coexist in the same app: AI SDK streaming for chat (where everything is tokens), NDJSON for orchestration (where events have heterogeneous shapes).

Backpressure

Both stream patterns handle backpressure correctly without explicit code. **controller.enqueue** in NDJSON respects the consumer's read pace via the underlying ReadableStream. The AI SDK's **streamText** respects the same. If the client is slow to read, the server pauses naturally; if the connection drops, the server-side promise rejects and the cleanup runs.

Failure modes

Three failure modes are worth naming.

The model errors mid-stream. The AI SDK propagates the error as a final stream event; NDJSON endpoints emit **{ type: "error", data: { message } }** before closing. The UI catches both, surfaces a toast, and offers a retry button.

The user navigates away mid-run. The fetch is aborted by the browser. The server-side **AbortSignal** fires, which triggers cleanup — the run row is marked **cancelled**, partial state is discarded. The user can return to the notebook later and start a fresh run; nothing is left in a half-state.

The connection drops. The fetch rejects with a network error on the client. The UI surfaces "connection lost" and offers retry. There is no automatic reconnection — for a one-shot run, the right behaviour is to start over from a known state, not to silently resume into ambiguity.

Why this is the right shape for agent UIs

Agentic systems do work. The work takes time. The user wants to know what is happening. Streaming + AG-UI gives them all three: real-time visibility, structured progress, and a contract that says "every step the agent takes will appear here."

The contract is what makes users trust the system. Without it, the agent is a black box that takes 90 seconds to return a report; with it, the agent is a visible thinking partner whose process is observable. The same model produces the same report in both cases. The product feels different because the experience around it is different.

Streaming is not a feature. It is a forcing function. If the endpoint must stream, the work shape becomes "observable events," and the UI gets progress for free.

THE STUDIO PANE • EIGHT OUTPUT KINDS, ONE GENERATOR

13

Audio, mind maps, *flashcards*, quizzes.

The studio is where every generated artifact lives. Eight different kinds, all backed by the same **studio_outputs** table, all generated through the same tRPC **studio.generate** mutation, all rendered through their own modal viewer. The shape of the artifact differs; the lifecycle is shared. Five pages, one per kind family.

The eight kinds

Audio overview

Two-host podcast script, TTS-rendered, MP3 download
script JSON • Deepgram Aura • Buffer.concat

Mind map

Markdown heading hierarchy rendered with markmap-lib
markdown • markmap • interactive zoom

Flashcards

15-25 front/back pairs, flippable
JSON [{ front, back }] • React state

Quiz

5-30 multiple-choice with answer index
JSON [{ question, options, answer }]

Study guide

Concepts, definitions, review questions
markdown • headings + bullets

Briefing doc

Executive summary, findings, recommendations
markdown • structured sections

FAQ

10-15 questions with detailed answers
markdown • Q/A blocks

Timeline

Chronological events with descriptions
markdown • ordered list

The single generator

All eight kinds (except audio overview, which is a streaming route) flow through one tRPC mutation. The mutation reads ready sources from the notebook, concatenates their content up to a 20,000-character cap, builds a kind-specific prompt, runs **generateText**, and stores the result on the **studio_outputs** row.

```
// src/server/routers/studio.ts - single generator for 7 of the 8 kinds
const readySources = await db.select({ content: sources.content })
  .from(sources).where(and(eq(sources.notebookId, id), eq(sources.status, "ready")));

let combinedContent = "";
for (const s of readySources) {
  combinedContent += s.content + "\n\n";
  if (combinedContent.length ≥ 20000) break;
}

const prompt = buildPrompt(input.kind, combinedContent, { questionCount: input.questionCount });
const { text } = await generateText({
  model: await getChatModel(ctx.user.id),
  prompt,
});

if (input.kind === "mind-map") content = { markdown: stripFences(text) };
else if (STRUCTURED_KINDS.has(input.kind)) content = JSON.parse(stripFences(text));
else content = { text };
```

The 20,000-character cap is what fits comfortably in a Gemini Flash prompt alongside the kind-specific instructions and Zod schema. For longer notebooks, the cap means the generator only sees the beginning of each source. A v2 enhancement would summarise sources first and pass the summaries; for now, the cap is the constraint that keeps token costs predictable.

Audio overview — the two-voice trick

The audio overview is the studio's flagship output and the only one that gets a streaming endpoint of its own. It runs four stages, all visible to the user as the audio overview modal streams progress events.

STAGE 1 · SCRIPT GENERATION

```
// Gemini Flash streams a JSON array of { speaker, text } segments
const { partialObjectStream } = streamObject({
  model: chatModel,
  schema: z.object({ segments: z.array(z.object({ speaker: z.enum(["Alex", "Sam"]), text: z.string() })) }),
  prompt: audioScriptPrompt(combinedContent),
});

for await (const partial of partialObjectStream) {
  send("script-delta", { segments: partial.segments ?? [] });
}
```

The script streams as it is written, so the user watches the conversation come into existence. Each **script-delta** event re-renders the script preview in the modal, which is essential for keeping the user engaged across the 30-60 seconds it takes to write a 12-segment script.

The script prompt does the work. It tells the model that Alex is the explainer who introduces ideas; Sam is the skeptical interlocutor who asks follow-up questions and adds reactions. The single most important instruction in the prompt — the one that turns "two monologues taking turns" into "an actual podcast" — is *"Sam should occasionally say 'That's fascinating' or 'Wait, so you're saying...'"*. That instruction adds the conversational glue that makes the audio feel like a real exchange rather than a scripted recitation.

STAGE 2 · TWO-VOICE TTS IN PARALLEL BATCHES

```
const VOICE_MAP: Record<string, string> = {
  Alex: "aura-orion-en",    // warm male
  Sam:  "aura-asteria-en",  // warm female
};

async function ttsSegment(seg: Segment): Promise<Buffer> {
  const voice = VOICE_MAP[seg.speaker] ?? "aura-asteria-en";
  const res = await fetch(`https://api.deepgram.com/v1/speak?model=${voice}&encoding=mp3`, {
    method: "POST",
    headers: { Authorization: `Token ${apiKey}`, "Content-Type": "application/json" },
    body: JSON.stringify({ text: seg.text }),
  });
  return Buffer.from(await res.arrayBuffer());
}

// Render four segments in parallel, repeat until done
for (const batch of chunkArray(script.segments, 4)) {
  const buffers = await Promise.all(batch.map(ttsSegment));
  audioBuffers.push(...buffers);
  send("tts", { done: audioBuffers.length, total: script.segments.length });
}
```

STAGE 3 · CONCATENATION

Plain **Buffer.concat(audioBuffers)**. No FFmpeg, no crossfades, no stitching cleverness. MP3 is forgiving about naive concatenation — frame boundaries align cleanly enough that the result plays without audible glitches in every player I have tested. It is not broadcast-perfect, but it sounds like a podcast.

STAGE 4 · UPLOAD AND PERSIST

The combined buffer goes to Supabase Storage. The public URL is written to the **studio_outputs.assetUrl** column. The studio panel shows the audio card with a play button; clicking it opens an HTML5 audio player in the modal viewer.

The quality is all in the script. Deepgram Aura voices are good enough that the listener's brain stops noticing them about thirty seconds in. What they notice is whether the two hosts actually have a conversation — whether Sam asks real questions, whether Alex has room to explain, whether there is rhythm. That is prompt engineering, not audio engineering.

Mind map — markmap-driven SVG

The mind map is a markdown heading hierarchy rendered as an interactive SVG via **markmap-lib**. The model produces the markdown; markmap renders it.

THE PROMPT

```
const prompt = `Generate a mind map in Markdown heading format for use with the markmap library.

Rules:
- Use a single # heading for the central topic.
- Use ## for main branches (aim for 4-7 branches).
- Use ### for sub-topics under each branch (2-5 per branch).
- Use #### for further details where appropriate (1-3 per sub-topic).
- Keep each node text concise (max 6-8 words).
- Cover ALL the major concepts from the source material.
- Do NOT include any fenced code blocks, do NOT wrap in backticks.
- Return ONLY the markdown, nothing else.

Example format:
# Artificial Intelligence
## Machine Learning
### Supervised Learning
#### Classification
#### Regression
### Unsupervised Learning
## Deep Learning
### Neural Networks
### CNNs

Source Material:
${combinedContent}`;
```

The constraint set is what produces a good mind map every time: one root, 4-7 branches, 2-5 sub-topics per branch, optional 1-3 details. The result is balanced, readable, and renders cleanly without extreme depth or width imbalances.

THE RENDERER

```
// src/module/notebook/components/MindMapRenderer.tsx
import { Markmap } from "markmap-lib";
import { Transformer } from "markmap-lib";

const transformer = new Transformer();
const { root } = transformer.transform(markdown);
const mm = Markmap.create(svgRef.current, { duration: 300, maxWidth: 320 }, root);

// User can pan, zoom, collapse branches by clicking nodes
```

The renderer takes the markdown, transforms it to a tree structure, and hands it to markmap which produces an interactive D3-backed SVG. The user can pan, zoom, collapse and expand branches by clicking. Resizing the modal re-flows the layout. The result is a mind map that feels like a real product, not a static image.

No additional logic in our code — markmap does the heavy lifting. We compose the model's output (clean markdown headings) with markmap's rendering (interactive SVG) to get a feature that would otherwise have been a week of custom drawing code.

Flashcards — JSON in, flippable cards out

```
// Flashcard prompt produces structured JSON
prompt = base + 'Generate flashcards as a JSON array with the following structure: ' +
  ' [{ "front": "question or term", "back": "answer or definition" } ]. ' +
  ' Create 15-25 flashcards covering the key concepts. Return ONLY valid JSON.';

// After generation, JSON.parse and store
content = JSON.parse(stripFences(generatedText));
// content is [{ front, back }, { front, back }, ...]
```

The flashcard viewer is one component. ~90 lines. State holds the current card index and a flipped boolean. Click flips the card. Arrow keys navigate. Spacebar flips. Card counter shows "5 of 22" at the top. A "shuffle" button randomises order. A "study mode" button hides cards the user has marked correct.

Quiz — multiple-choice with scoring

```
// Quiz prompt
prompt = base + 'Generate a quiz as a JSON array with the following structure: ' +
  ' [{ "question": "...", "options": ["A", "B", "C", "D"], "answer": 0 } ] ' +
  `where answer is the zero-based index of the correct option. Create exactly ${count} questions. Return ONLY valid JSON.`;
```

The quiz config modal lets the user pick the question count (5–30) before generating. The default is 10. The kind-specific tRPC input includes **questionCount** as an optional integer.

The quiz viewer renders one question at a time with the four options as radio buttons. Clicking an option selects it; a "next" button advances. After the last question, the viewer scores the responses and shows a summary: total correct, total questions, percentage, and per-question breakdown highlighting which were right and which were wrong (with the correct answer revealed).

A subtle but important UX choice: the user can review their wrong answers and click each one to read a generated explanation. The explanation is produced by a separate streamText call to **/api/studio/quiz-summary** — that is the second streaming endpoint in the studio. The explanation streams in token-by-token, which keeps the review session feeling alive.

Study guide, briefing, FAQ, timeline — markdown variants

The remaining four kinds — study guide, briefing doc, FAQ, timeline — are all markdown-output kinds. They use the same generator path, the same generateText call, the same content shape (**{ text }**). The difference is purely in the prompt.

Study guide

"Generate a comprehensive study guide with key concepts, definitions, review questions, and summaries..."

prompt: structured • output: markdown

Briefing doc

"Generate an executive briefing. Include executive summary, key findings, analysis, implications, recommendations..."

prompt: executive tone • output: markdown

FAQ

"Generate a FAQ with 10–15 questions and detailed answers based on the source material..."

prompt: Q/A pairs • output: markdown

Timeline

"Generate a chronological timeline of key events. Include dates and brief descriptions..."

prompt: chronological • output: markdown

The renderer for all four is the same — a markdown component that uses **react-markdown** with custom styles for headings, bullets, blockquotes, and code. The markdown is rendered the same way whether it is a study guide or a briefing doc; only the content differs.

Notes — user-authored, same lifecycle

Notes are studio outputs too, but they are not LLM-generated. The user creates them inline in the studio panel via a "new note" button that opens a markdown editor. The note's content is saved as **{ markdown }** in the same **studio_outputs.content** JSONB column. The kind is **"note"**.

Why include user-authored notes in the studio? Three reasons. First, it gives the user a single place to keep their thoughts about a notebook alongside the AI-generated artifacts. Second, it lets us treat a note as a source for the chat — if the user types **@my-notes** in the composer, the chat can scope retrieval to include their notes. Third, it makes the studio the canonical "everything I have made about this notebook" surface, which is the right mental model.

The unified data model

All eight kinds live in **studio_outputs** with kind, title, content (JSONB), assetUrl (for audio), status, and timestamps. One table, eight kinds. Adding a ninth kind is one row added to **KIND_TITLES**, one branch in **buildPrompt**, one renderer component in the modal viewer. Twenty minutes of work to add a new kind.

One database table, one tRPC mutation, one switch on kind. Eight different artifacts. The studio is what happens when you generalise correctly the first time.

The studio panel UX

The studio panel on the right of the workbench shows every output for the current notebook as a card, ordered by creation time descending. Each card has the kind icon, the title, the status badge, and a small action menu (open, regenerate, delete).

Clicking a card opens **StudioOutputModal** at full screen with the right view component for the kind: the audio player for audio overviews, **MindMapRenderer** for mind maps, the flashcard flipper for flashcards, the quiz player for quizzes, the markdown renderer for the four markdown kinds, and the note editor for notes.

The "generate" flow

The generate button at the top of the studio panel opens a chooser of the eight kinds. Each kind in the chooser shows its name, a one-line description, and a small icon. Picking a kind kicks off either the tRPC **studio.generate** mutation (for the seven non-streaming kinds) or the route handler call (for audio overview).

For the non-streaming kinds, the row appears immediately in the panel with status **generating**, then transitions to **ready** when the mutation completes. The transition is announced via tRPC's optimistic-update flow — the row updates in the local cache before the server's response, then reconciles when the response arrives.

For audio overviews, picking the kind opens **AudioOverviewModal** directly, which kicks off the streaming endpoint and shows the AG-UI events live. When the audio is finished, the modal stays open with the audio player; closing it returns to the workbench with the audio card now in the studio panel.

Regenerate

Every studio output has a "regenerate" action. Clicking it kicks off the same generation flow with the same kind and the same source content (whatever sources are **ready** at the time of regeneration). The new output replaces the old one — same row id, new content, status returns to **generating** and back to **ready**.

Why replace instead of creating a new row? Because users typically regenerate when they were not satisfied with the first attempt. Keeping the old version around clutters the panel. If a user wants both, they can do it manually — copy the first output, then regenerate.

What I would add later

Five enhancements are on the list. **Per-output source filtering** — currently the generator uses every ready source; it would be nice to scope a study guide to just two specific papers. **Custom kinds** — let users define their own kind with a custom prompt. **Output sharing** — public link for an audio overview or mind map. **Voice selection** — let users pick the two TTS voices for their audio overview. **Background generation** — kick off five different kinds simultaneously and have them all populate the panel as they finish.

None of these is hard; all of them are downstream of the foundation that already exists. The current studio is the v1 that proved the unified shape works. Each enhancement is a small extension of the same model.

The shape this gives the product

A notebook is not just a chat. It is a workbench where every artifact the user makes — researched, generated, or hand-authored — lives in one place with one lifecycle and one viewer. The studio pane is the home for that body of work. As the notebook accumulates audio overviews, mind maps, flashcards, study guides, and notes, the studio becomes the user's record of how they engaged with the material.

Most "AI app" UIs do not have this. Most are chat-only with maybe one or two output kinds bolted on. The studio pattern is what turns NotebookLM-style products from "AI search engine" into "AI workbench." It is the single highest-leverage UI move in the entire build.

MULTI-PROVIDER AI • TWELVE PROVIDERS, ONE FACTORY

Twelve providers, *encrypted* at rest.

14

A user signs into the app, picks any of twelve AI providers, drops in their API key, picks a model, and uses the product end to end. The key never leaves the database in plaintext. The factory swaps providers without code changes. This chapter is how — the registry, the encryption model, the factory dispatch, the test-connection flow, and the onboarding gate that ties it all together.

THE PROVIDER REGISTRY

OpenAI gpt-4o • o1 • embed-3-small/large 1536 / 3072	Anthropic Claude 4.5 Opus / Sonnet / Haiku chat only	Google Gemini 2.5 Pro / Flash • embed-001 768 (truncated)
Mistral Large • Medium • Codestral • embed 1024	Cohere Command A • embed v3 384 / 1024	Voyage AI voyage-3 • voyage-code-3 512 / 1024
Groq Llama 3.3 • Mixtral • Gemma chat only	Ollama local • custom • embed 768 / 1024	OpenRouter aggregator • custom models via base provider
Together AI Llama Turbo • BGE • UAE 768 / 1024	xAI Grok 2 • Grok Beta chat only	OpenAI-compatible any custom OpenAI-shaped endpoint user-defined

The registry as the source of truth

All twelve providers are defined in one file: **src/lib/ai/providers.ts**. Each provider has an id, a label, a logo path, an auth type (**api_key**, **api_key_and_base_url**, or **base_url_only**), and a list of model definitions. Each model has an id, a label, a list of capabilities (**"chat"** or **"embed"**), and either a **contextWindow** (for chat models) or an **embedDim** (for embedding models).

The registry is the single source of truth for what shows up in the settings dashboard. Adding a new provider is one entry in the array. Adding a new model under an existing provider is one entry in the provider's models list. There is no other place in the codebase that knows about provider catalogues.

Three providers are special. **Ollama** has **baseUrl** required (default localhost:11434), **selfHostedOnly: true**, and **supportsCustomModels: true** — users can type model ids that we do not list. **OpenRouter** and **openai_compatible** have **supportsCustomModels: true** for the same reason — they are aggregators or proxies, and the model space is open-ended.

HELPER FUNCTIONS

```
export function chatProviders(): ProviderDef[] {
  return PROVIDERS.filter(p =>
    p.supportsCustomModels || p.models.some(m => m.capabilities.includes("chat"))
  );
}
export function embedProviders(): ProviderDef[] { /* same shape */ }
export function chatModels(p: ProviderId): ModelDef[] { /* filter by capability */ }
export function embedModels(p: ProviderId): ModelDef[] { /* filter by capability */ }
```

The settings page calls **chatProviders** to populate its chat-provider dropdown, and **embedProviders** for the embedding dropdown. When the user picks a provider, **chatModels** or **embedModels** populates the model dropdown for that provider. The registry drives the entire settings UI.

The encrypted credential vault

User API keys are stored in **user_provider_credentials** as AES-GCM ciphertext. The columns: **apiKeyCiphertext**, **apiKeyIv**, **apiKeyTag**, **apiKeyKeyVersion** — all base64. Encryption uses the **userId** as additional authenticated data (AAD), so a ciphertext from one user's row cannot be decrypted under another user's identity even if both rows used the same key version.

```
// src/lib/crypto/secret.ts - encrypt
export function encryptSecret(plaintext: string, aad: string): EncryptedSecret {
  const iv = crypto.randomBytes(12);
  const key = getActiveKey(); // 32 bytes from MASTER_KEY env
  const cipher = crypto.createCipheriv("aes-256-gcm", key, iv);
  cipher.setAAD(Buffer.from(aad, "utf8"));
  const ciphertext = Buffer.concat([cipher.update(plaintext, "utf8"), cipher.final()]);
  return {
    ciphertext, iv, tag: cipher.getAuthTag(),
    keyVersion: ACTIVE_KEY_VERSION,
  };
};

// Decrypt - userId becomes part of the auth tag
export function decryptSecret(secret: EncryptedSecret, aad: string): string {
  const key = getKeyForVersion(secret.keyVersion);
  const decipher = crypto.createDecipheriv("aes-256-gcm", key, secret.iv);
  decipher.setAAD(Buffer.from(aad, "utf8"));
  decipher.setAuthTag(secret.tag);
  return decipher.update(secret.ciphertext) + decipher.final("utf8");
}
```

Key versioning is built in. When the master key rotates, new credentials are encrypted under the new version, but old credentials remain readable until they are re-saved. Rotation is gradual and non-disruptive. The plaintext API key only ever exists in the running process for the duration of the LLM call.

The factory

```
// src/lib/ai/factory.ts - getChatModel for an authenticated user
export async function getChatModel(userId: string): Promise<LanguageModel> {
  const [cfg] = await db.select().from(userAiConfig)
    .where(eq(userAiConfig.userId, userId)).limit(1);

  if (!cfg?.chatProvider || !cfg?.chatModel) throw new NoAiConfigError("chat");

  const cacheKey = `${userId}:chat:${cfg.updatedAt.getTime()}:${cfg.chatProvider}:${cfg.chatModel}`;
  const cached = chatCache.get(cacheKey);
  if (cached) return cached;

  const credential = await loadCredential(userId, cfg.chatProvider);
  const model = buildChatModel(cfg.chatProvider, cfg.chatModel, credential.apiKey, credential.baseUrl);
  chatCache.set(cacheKey, model);
  return model;
}
```

The factory is the seam between "the user's configured provider and key" and "an AI SDK LanguageModel ready to call." It loads the user's config, loads and decrypts the credential, dispatches to the right AI SDK constructor, and caches the result keyed by user+model+config-mtime so subsequent calls within 10 minutes skip the database round-trip.

The cache is bounded (max 500 entries) and TTL-bounded (10 minutes). Settings changes invalidate the cache by updating **userAiConfig.updatedAt**, which changes the cache key. Manual invalidation is also exposed via **invalidateUserAiCache(userId)** for the credential update flow.

Provider dispatch

```
function buildChatModel(provider: ProviderId, modelId: string, apiKey?: string, baseUrl?: string | null): LanguageModel {
  const baseUrl = baseUrl ?? undefined;
  switch (provider) {
    case "openai": return createOpenAI({ apiKey, baseUrl })(modelId);
    case "anthropic": return createAnthropic({ apiKey })(modelId);
    case "google": return createGoogleGenerativeAI({ apiKey })(modelId);
    case "mistral": return createMistral({ apiKey, baseUrl })(modelId);
    case "cohere": return createCohere({ apiKey, baseUrl })(modelId);
    case "groq": return createGroq({ apiKey, baseUrl })(modelId);
    case "xai": return createXai({ apiKey, baseUrl })(modelId);
    case "together": return createTogetherAI({ apiKey, baseUrl })(modelId);
    case "openrouter":
    case "ollama":
    case "openai_compatible": {
      const fallbackBase = getProvider(provider)?.defaultBaseUrl;
      const compat = createOpenAICompatible({
        name: provider,
        apiKey: apiKey ?? "",
        baseUrl: baseUrl ?? fallbackBase ?? "",
      });
      return compat(modelId);
    }
    case "voyage": throw new Error("Voyage does not support chat");
  }
}
```

Twelve cases. Eleven of them are a single line each. Three providers (OpenRouter, Ollama, openai_compatible) collapse into the OpenAI-compatible adapter because their wire formats are OpenAI-shaped. Voyage explicitly errors because Voyage is embeddings-only; trying to use it as a chat provider is a configuration mistake we want to catch loudly.

Every case returns a **LanguageModel**. Every consumer of the factory — chat, deep research, studio, retrieval rerank — calls the same **generateText**, **generateObject**, or **streamText** on whatever **LanguageModel** comes back. None of them know which provider is in use. Switching a user from Gemini Flash to Claude Sonnet is one row update on **userAiConfig**. Zero code changes.

The embedding factory

Embeddings have a parallel factory: **getEmbedFn(userId)** returns an **EmbedHandle** with **embed(texts)**, **dim**, **model**, and **provider** fields. The handle is provider-agnostic — call **handle.embed([text])** and get back a 768-dim or 1024-dim or 1536-dim or 3072-dim vector depending on which provider and model is configured. The dim travels on the handle so the rest of the system knows which sibling table to write into.

The dispatch is similar to the chat factory but with adapter-shaped functions per provider. Each adapter knows how to call its provider's embedding endpoint and how to handle provider-specific quirks (e.g. Google's **outputDimensionality** parameter for truncated dims).

Test connection

Before saving a credential, the settings page calls **provider.testConnection** with the unsaved key. The factory has companion functions — **buildChatModelFromParams**, **buildEmbedHandleFromParams** — that bypass the database and run a tiny sample call (one-token completion for chat, one-text embedding for embed). The result is "ok," "invalid," "rate-limited," or "unknown." Users get instant feedback before committing the key.

The test-connection flow also doubles as an early-warning system for misconfigured base URLs (Ollama, OpenAI-compat) — if the URL is wrong, the test fails immediately rather than failing silently when the user later tries to chat.

Why the AI SDK is the whole thing

The factory is what lets one notebook product run on twelve providers without forking the codebase. The AI SDK is what makes the factory possible, by exposing a uniform **LanguageModel** interface across every provider's SDK. Without the AI SDK, the factory would be a Frankenstein of provider-specific call shapes; with it, every consumer of a model just calls **generateText({ model, prompt })** and the SDK handles the rest.

One Postgres column says which provider, one says which model. One factory turns that into a model handle. One uniform interface lets every consumer ignore which provider is in use. Provider portability is not a feature you build. It is what falls out of using the right primitive.

Every claim, traced to *a chunk*.

The reason NotebookLM is taken seriously by researchers and ChatGPT mostly is not, despite ChatGPT being the more capable model, is that NotebookLM cites and ChatGPT does not. Citations are the feature that turns AI output into something a professional will use.

The mechanism

After the three-stage retrieval, the top **k** chunks go into the chat system prompt, each numbered **[1]**, **[2]**, **[3]**... up to **[k]**. The model is told, in the system prompt: *"cite every non-trivial claim with [^n] markers where n is the source number below."* The chat model produces text that includes those markers inline.

```
// src/app/api/chat/route.ts - system prompt construction
const systemPrompt = `You are a research assistant that answers strictly from the provided sources.

- Cite every non-trivial claim with [^n] markers where n is the source number below.
- If the sources don't contain the answer, say so - don't invent facts.
- Be concise and structured.

SOURCES:
${retrieved.map((r, i) => `[${i + 1}] ${r.sourceTitle}\n${r.content}`).join("\n\n")}`;
```

A real example

The chunker splits text on markdown headings before falling back to paragraphs^[^1], and uses sentence overlap rather than character overlap to maintain context across chunk boundaries^[^2]. Embeddings are produced through the gemini-embedding-001 endpoint, with **outputDimensionality** set to 768 to match the pgvector column dimension^[^3].

[^1] src/lib/ingest/chunk.ts · "Heading-aware splits flush the current chunk and carry the heading forward..."

[^2] src/lib/ingest/chunk.ts · "When a chunk closes, the last sentence of the previous chunk seeds the next..."

[^3] src/lib/ingest/embed.ts · "The AI SDK does not pass outputDimensionality through, so we POST directly..."

The persistence

When the streamed response closes, the **onFinish** callback persists the message *with its citations* as a JSONB column on the **messages** table:

```
await db.insert(messages).values({
  notebookId, role: "assistant", content: text,
  citations: retrieved.map((r, i) => ({
    n: i + 1,
    chunkId: r.chunkId,
    sourceId: r.sourceId,
    title: r.sourceTitle,
    snippet: r.content.slice(0, 240),
  })),
});
```

The citation snippet is frozen at the moment of generation. Re-opening the conversation later does not require re-running retrieval; the message and its citations are co-stored. If the underlying chunk is deleted (because the source was removed), the citation row still has the snippet — readers can see what the AI was looking at even if the original chunk is gone.

This is more honest than the alternative — re-running retrieval at fetch time and surfacing whatever chunks are currently best matches. The frozen citations are the actual evidence the AI used; live citations would be fiction we constructed after the fact.

The UI layer

In the chat panel, the regex `/\[\^\(\d+)\]/g` matches the citation markers in the streamed text. Each match is replaced by a clickable superscript pill showing the citation number. Clicking the pill opens a tooltip card with the source title, the snippet (240 characters), the kind icon, the tag chips, and a link that scrolls to the source in the left pane (or opens the source modal if it is a file). The tooltip is also keyboard-accessible — focusing the pill with Tab and pressing Enter opens it.

```
// In the message renderer
function renderText(text: string, citations: Citation[]) {
  const parts = text.split(/[\[\^\(\d+)\]/g);
  return parts.map((part, i) => {
    const match = part.match(/^\[\^\(\d+)\]$/);
    if (!match) return <span key={i}>{part}</span>;
    const n = parseInt(match[1], 10);
    const cite = citations.find(c => c.n === n);
    return <CitationPill key={i} citation={cite} />;
  });
}
```

Tags and source kinds

Sources can be tagged in the metadata column. Tags appear as small chips next to the source title in the panel. The chat retrieval can be scoped to specific source ids, and the citation tooltip shows the source's tag chips so the reader knows whether they are looking at a paper, a transcript, or their own notes.

The kind column distinguishes how the source originated (**file**, **link**, **text**, **audio**) and surfaces as an icon in the citation pill. A pill that points to a paper looks different from a pill that points to a web link or a research report. The visual distinction lets the reader assess source quality at a glance without opening the tooltip.

Multi-source confidence (deep research)

Deep research adds an extra layer on top of basic citations. The verification step (Chapter 11) emits a **verifiedClaims** array with each major claim, the source numbers that support it, and a confidence label (**high**, **medium**, **low**). The UI surfaces this as confidence badges next to factual claims in the report — green dot for high, amber for medium, coral for low.

A claim supported by three or more sources is marked high-confidence. A claim supported by two is medium. A claim supported by only one is low — flagged as "single-sourced, treat with caution." Contradictions between sources surface as warnings in the sources panel. The user does not have to manually cross-reference; the system has already done it.

When the AI cannot answer from sources

A nice consequence of the citation discipline: when the sources do not contain the answer to a question, the model says so explicitly rather than making something up. The system prompt instructs *"if the sources don't contain the answer, say so — don't invent facts,"* and Gemini Flash respects this instruction reliably.

A response like *"The provided sources do not directly address this question. They cover X and Y, but not Z."* is more honest than a confident-sounding hallucination. Users learn to trust the system's "I don't know" because it is well-calibrated. That trust is itself a feature, and it traces back to the same citation pipeline that produces the bracket markers.

What citations are not

Worth being explicit about the boundary. Citations in this system are *which chunks the model saw*, not *which claims the model factually verified*. The model could still get things wrong even when citing — it could quote a chunk but misinterpret its meaning, or cite chunk X for a claim that actually came from chunk Y in its retrieval set. The citations narrow the reader's verification window from "the entire document" to "this 240-character snippet"; they do not eliminate the need for verification entirely.

For most users this distinction does not matter much — they spot-check a few citations and move on. For high-stakes use cases (legal, medical, regulatory) it would matter, and the right v2 enhancement is a "verify this claim" button that asks the model to extract the exact phrase from the cited chunk that supports the claim. That is a future feature; the current citations are already the difference between AI output that is usable and AI output that is not.

Citations are not garnish. They are the contract that makes the rest of the AI usable. A researcher cannot trust output they cannot verify. The retrieval pipeline produces the chunks, the system prompt converts them to numbered references, the model emits inline markers, the persistence freezes them with the answer, and the UI surfaces them as clickable evidence. Five layers, one

Same way everytime.

Forty-five pages in. Here is the honest balance sheet — the decisions I would repeat unchanged, and the single sentence that compresses the entire build.

SAME WAY EVERY TIME

- Route Handlers for streaming, tRPC for everything else, Drizzle types end-to-end, Better Auth for sessions, pgvector on the same Postgres. The whole architecture fits in working memory.
- Every provider returns a LanguageModel. streamText, generateObject, generateText accept any of them identically. Switching providers is one row in user_ai_config.
- Query expansion · hybrid vector + keyword · LLM rerank. ~700ms per question. Lifts retrieval from "sometimes finds the right thing" to "almost always finds the right thing."
- Plan, search, draft, score 1-10, gap-fill if needed, fact-verify. The agent grades itself. Reports are measurably more grounded than a single-pass write.
- Sibling tables per dim. Switching embedding models is non-destructive. Old vectors stay until backfilled. Database evolution as additive change.
- One line per event. Every step is a UI event. The user watches the work happen instead of waiting for a black box to return.
- AES-GCM, userId as AAD, key versioning baked in. Plaintext API keys never live in the database. Twelve providers, one vault.
- Frozen with the answer at generation time. Rehydrating a conversation does not require re-running retrieval. Provenance is a product feature.
- Eight kinds, one table, one mutation. Audio overviews, mind maps, flashcards, quizzes, study guides, briefings, FAQs, timelines, notes — all first-class.

The NotebookLM-feel is not a model. It is a retrieval pipeline that does three stages, a research loop that critiques itself, two TTS voices that take turns, and a workbench that gives every artifact a home. Every one of those is a few hundred lines of plain TypeScript against models you are already paying for.

Written by *Halleluyah Darasimi Oludele* · Built between 2026-04-21 and 2026-04-24 · Field Notes Issue 01 · github.com/hallelx2/notebooklm-web · Open notes on building with agents